



UNIVERSIDAD DE GRANADA
MÁSTER EN INGENIERÍA INFORMÁTICA

TRABAJO FIN DE MÁSTER

Desarrollo de una aplicación de gestión de planes de boda

Presentado por:
Laura Rosón Lavín

Tutor:
Juan José Ramos

Curso académico 2025 / 2026

D./Dña.: Juan José Ramos Muñoz, tutor/a del trabajo titulado **Desarrollo de una aplicación de gestión de planes de boda** realizado por el alumno/a **Laura Rosón Lavín**, INFORMA que dicho trabajo cumple con los requisitos exigidos por el Reglamento sobre Trabajos Fin de Máster en Ingeniería Informática para su defensa, y autorizo su presentación.

Granada, 4 de Septiembre de 2026

Fdo.: _____

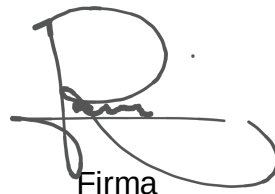
Por la presente dejo constancia de ser el/la autor/a del trabajo titulado **Desarrollo de una aplicación de gestión de planes de boda** que presento para la materia Trabajo Fin de Máster del Máster en Ingeniería Informática, tutorizado por el/la profesor/a **Juan José Ramos Muñoz** durante el curso académico 2025- 2026.

Asumo la originalidad del trabajo y declaro que no he utilizado fuentes (tablas, textos, imágenes, medios audiovisuales, datos y software) sin citar debidamente, quedando la Escuela Internacional de Posgrado de la Universidad de Granada exenta de toda obligación al respecto.

Autorizo a la Escuela Internacional de Posgrado a utilizar este material para ser consultado con fines docentes dado que constituyen ejercicios académicos de uso interno.

03 / 09 / 2026

Fecha

A handwritten signature in black ink, consisting of a large, stylized capital letter 'R' with a horizontal line extending from its base and a small loop at the end.

Firma

RESUMEN

La organización de una boda es una tarea compleja y de larga duración en la que intervienen numerosos aspectos interdependientes —invitados, mesas, tareas, presupuesto y proveedores— que habitualmente se gestionan de forma fragmentada y propensa a errores. Este Trabajo Fin de Máster presenta el desarrollo de una aplicación web que permite organizar de forma integral y centralizada todos estos aspectos desde un único panel, e incorpora un conjunto de asistencias basadas en inteligencia artificial que automatizan las decisiones organizativas más tediosas.

La aplicación permite gestionar invitados y acompañantes con seguimiento de confirmaciones de asistencia (RSVP) y envío de invitaciones por correo electrónico, agrupar invitados y distribuirlos en las mesas del banquete, planificar tareas y eventos, controlar el presupuesto y los proveedores —con documentos adjuntos— y consultar un panel de seguimiento con avisos sobre tareas y pagos próximos. Un mismo usuario puede gestionar varias bodas de forma aislada y segura.

El sistema se ha desarrollado siguiendo una arquitectura cliente-servidor: una interfaz construida con React y TypeScript, y un servidor Node.js con Express que expone una API (interfaz de programación de aplicaciones) REST y accede a una base de datos PostgreSQL mediante el ORM (mapeador objeto-relacional) Prisma. La autenticación se basa en tokens JWT (JSON Web Tokens) con renovación de sesión, y la autorización, en la propiedad de cada boda. Las asistencias de inteligencia artificial, apoyadas en la API de OpenAI e invocadas exclusivamente desde el servidor, proponen listados de tareas, distribuciones de mesas y repartos de presupuesto en forma de propuestas editables que el usuario revisa y confirma antes de aplicarlas.

El resultado es una herramienta funcional y autocontenida que cubre el ciclo completo de la organización de una boda.

Palabras clave: planificación de bodas, aplicación web, inteligencia artificial, gestión de invitados, distribución de mesas, desarrollo full-stack.

ABSTRACT

Planning a wedding is a complex, long-running task involving many interdependent aspects — guests, tables, tasks, budget and vendors — that are usually managed in a fragmented and error-prone way. This Master's Thesis presents the development of a web application that allows all these aspects to be organised in an integrated and centralised manner from a single dashboard, and incorporates a set of artificial-intelligence assistants that automate the most tedious organisational decisions.

The application makes it possible to manage guests and their companions with RSVP tracking and email invitations, to group guests and arrange them at the banquet tables, to plan tasks and events, to control the budget and vendors —with attached documents— and to consult a tracking dashboard with reminders about upcoming tasks and payments. A single user can manage several weddings in an isolated and secure way.

The system was developed following a client-server architecture: a user interface built with React and TypeScript, and a Node.js server with Express that exposes a REST API and accesses a PostgreSQL database through the Prisma ORM. Authentication is based on JWT tokens with session renewal, and authorisation on the ownership of each wedding. The artificial- intelligence assistants, powered by the OpenAI API and invoked exclusively from the server, propose task lists, table arrangements and budget allocations as editable proposals that the user reviews and confirms before applying them.

The result is a functional, self-contained tool that covers the full cycle of organising a wedding.

Keywords: wedding planning, web application, artificial intelligence, guest management, seating arrangement, full-stack development.

ÍNDICE

1. INTRODUCCIÓN

- 1.1. Contexto y motivación
- 1.2. Estructura de la memoria

2. ESTADO DEL ARTE

- 2.1. La problemática de organizar una boda
- 2.2. La digitalización de la planificación de eventos
- 2.3. Herramientas existentes
- 2.4. Limitaciones y oportunidad
- 2.5. Tecnologías y herramientas seleccionadas

3. OBJETIVOS

- 3.1. Objetivo general
- 3.2. Objetivos específicos
- 3.3. Relación de los objetivos con los requisitos

4. METODOLOGÍA

- 4.1. Metodología de desarrollo
- 4.2. Herramientas y entorno de trabajo
- 4.3. Organización del trabajo por iteraciones
- 4.4. Problemas encontrados durante el desarrollo

5. PLANIFICACIÓN Y ESTIMACIÓN DE COSTES

- 5.1. Planificación temporal
- 5.2. Estimación de costes

6. ANÁLISIS

- 6.1. Requisitos funcionales
- 6.2. Requisitos no funcionales
- 6.3. Actores
- 6.4. Casos de uso
- 6.5. Matriz de trazabilidad

7. DISEÑO

- 7.1. Arquitectura general
- 7.2. Modelo de datos
- 7.3. Comunicación y API REST
- 7.4. Autenticación y autorización
- 7.5. Estructura de navegación
- 7.6. Diseño de la interfaz

8. IMPLEMENTACIÓN

- 8.1. Estructura e inicialización del proyecto
- 8.2. Parte del cliente
- 8.3. Parte del servidor
- 8.4. Funcionalidades destacadas
- 8.5. Integración de la inteligencia artificial

9. PRUEBAS Y VALIDACIÓN

- 9.1. Estrategia de pruebas
- 9.2. Pruebas unitarias
- 9.3. Pruebas de integración
- 9.4. Resultados de las pruebas automatizadas
- 9.5. Pruebas funcionales manuales
- 9.6. Integración continua
- 9.7. Trazabilidad entre requisitos y pruebas

10. RESULTADOS

- 10.1. Grado de cumplimiento de los objetivos
- 10.2. Resultados de la aplicación
- 10.3. Resultados no previstos
- 10.4. Desviaciones y limitaciones

11. CONCLUSIONES Y LÍNEAS FUTURAS

- 11.1. Conclusiones
- 11.2. Líneas futuras

BIBLIOGRAFÍA

ANEXO A. MANUAL DE INSTALACIÓN

- A.1. Requisitos previos
- A.2. Obtención del código
- A.3. Puesta en marcha del backend
- A.4. Puesta en marcha del frontend
- A.5. Acceso a la aplicación
- A.6. Despliegue en producción

ANEXO B. MANUAL DE USUARIO

- B.1. Registro e inicio de sesión
- B.2. Panel de seguimiento
- B.3. Invitados
- B.4. Grupos
- B.5. Mesas

B.6. Tareas

B.7. Agenda

B.8. Presupuesto

B.9. Proveedores

B.10. Ajustes

B.11. Asistencias con inteligencia artificial

1. INTRODUCCIÓN

1.1. Contexto y motivación

La organización de una boda es una tarea compleja y de larga duración. En ella intervienen numerosos aspectos que deben coordinarse durante meses: la lista de invitados y el control de sus confirmaciones, la distribución de los invitados en las mesas del banquete, el reparto del presupuesto entre distintas categorías de gasto, la contratación y el seguimiento de proveedores y una larga sucesión de tareas con sus propios plazos. Estos elementos están relacionados entre sí. Por ello, resulta difícil mantener una visión de conjunto cuando la información se dispersa en hojas de cálculo, cuadernos o aplicaciones separadas. Según datos del sector, una boda implica de media unos 117 invitados y la contratación de alrededor de 13 profesionales distintos (The Knot Worldwide, 2026), lo que da una idea de la magnitud de la coordinación necesaria.

En la actualidad existen diversas plataformas que ofrecen herramientas para organizar una boda. Sin embargo, estas plataformas se construyen en torno a un modelo de negocio basado en el directorio de proveedores y el listado de regalos, del que obtienen sus ingresos. Sus herramientas de organización, aunque útiles, son un complemento de ese modelo. Además, la inteligencia artificial que han comenzado a incorporar se orienta principalmente a ese negocio —por ejemplo, la recomendación de proveedores— o a tareas de comunicación, y no a automatizar las decisiones organizativas internas de la pareja. Este análisis se desarrolla con detalle en el capítulo 2.

De esta situación surge la oportunidad que motiva el presente Trabajo Fin de Máster (TFM). La propuesta consiste en desarrollar una aplicación integrada y autocontenida, sin la capa de directorio comercial, en la que la inteligencia artificial se ponga al servicio de las decisiones más repetitivas de la organización, como planificar tareas, distribuir las mesas o repartir el presupuesto, siempre bajo el control de la pareja. De forma complementaria, el proyecto responde también a una motivación personal de la autora, para quien la organización de una boda constituye un objetivo cercano y para quien resultaba de interés dedicar el trabajo a una herramienta con una utilidad real.

En consecuencia, el objetivo general de este trabajo es desarrollar una aplicación web que permita organizar de forma integral y centralizada todos los aspectos de una boda, incorporando asistencias basadas en inteligencia artificial que automaticen las decisiones más repetitivas. Este objetivo se concreta en un conjunto de objetivos específicos que se detallan en el capítulo 3.

1.2. Estructura de la memoria

El resto de la memoria se organiza en los siguientes capítulos:

- El **capítulo 2 (Estado del arte)** sitúa el problema en su contexto, analiza las principales herramientas existentes para la organización de bodas y sus limitaciones, y presenta las tecnologías seleccionadas para el desarrollo.
- El **capítulo 3 (Objetivos)** define el objetivo general y los objetivos específicos del trabajo, y los relaciona con los requisitos del sistema.

- El **capítulo 4 (Metodología)** describe la metodología de desarrollo empleada, las herramientas de trabajo y los principales problemas encontrados durante el desarrollo.
- El **capítulo 5 (Planificación y estimación de costes)** presenta la planificación temporal del proyecto y la estimación de sus costes.
- El **capítulo 6 (Análisis)** especifica los requisitos funcionales y no funcionales, los actores y los casos de uso, junto con una matriz de trazabilidad.
- El **capítulo 7 (Diseño)** detalla la arquitectura del sistema, el modelo de datos, la comunicación entre cliente y servidor, la autenticación y autorización, la navegación y el diseño de la interfaz.
- El **capítulo 8 (Implementación)** explica cómo se ha llevado a la práctica el diseño, destacando las funcionalidades más relevantes y la integración de la inteligencia artificial.
- El **capítulo 9 (Pruebas y validación)** recoge las pruebas realizadas para verificar el correcto funcionamiento del sistema.
- El **capítulo 10 (Resultados)** evalúa el grado de cumplimiento de los objetivos y requisitos planteados.
- El **capítulo 11 (Conclusiones y líneas futuras)** resume lo conseguido, las limitaciones y las posibles vías de evolución del proyecto.

Finalmente, se incluye la **bibliografía** y una serie de **anexos** con el manual de instalación y el manual de usuario de la aplicación.

2. ESTADO DEL ARTE

Antes de abordar el desarrollo de la aplicación conviene situar el problema en su contexto: qué implica organizar una boda, cómo se ha digitalizado esa tarea, qué herramientas existen en la actualidad y qué limitaciones presentan. De ese análisis se deriva la oportunidad que este proyecto pretende cubrir. El capítulo se cierra con una descripción de las tecnologías y herramientas seleccionadas para la implementación, diferenciando así el **contexto del problema** de las **decisiones técnicas** que lo resuelven.

2.1. La problemática de organizar una boda

Organizar una boda es un proyecto complejo y de larga duración en el que intervienen numerosos aspectos interdependientes: la lista de invitados y el seguimiento de sus confirmaciones, la distribución de esos invitados en las mesas del banquete, el control de un presupuesto que se reparte entre múltiples categorías de gasto, la contratación y seguimiento de proveedores, y una larga lista de tareas que deben completarse en un orden y unos plazos determinados. Estos elementos no son independientes entre sí —el número de invitados condiciona las mesas, el presupuesto y los proveedores—, lo que multiplica la dificultad de mantener toda la información coherente y actualizada.

Tradicionalmente, esta coordinación se ha llevado con medios manuales (hojas de cálculo, cuadernos o documentos dispersos), un enfoque propenso a errores y difícil de mantener sincronizado. La propia estructura del dominio —una cuenta atrás de tareas organizadas por meses, un reparto del presupuesto por categorías o la vinculación entre invitados y mesas— se repite en las herramientas del sector, lo que evidencia que se trata de un problema bien acotado y susceptible de ser sistematizado mediante software.

2.2. La digitalización de la planificación de eventos

En los últimos años, la planificación de bodas se ha trasladado progresivamente de los medios manuales a **plataformas web y aplicaciones móviles** que centralizan la gestión. El mercado del software de planificación de bodas se estimó en torno a 1.500 millones de dólares en 2024, con una previsión de crecimiento sostenido en los próximos años (Growth Market Reports, 2025). La tendencia del sector apunta a disponer de un único «centro de mando» que reúna en un solo lugar todos los aspectos del evento: lista de invitados, mesas, presupuesto, tareas, proveedores y comunicación con los asistentes. Estas plataformas ofrecen sus servicios de planificación de forma gratuita para las parejas y obtienen sus ingresos de un modelo de mercado (*marketplace*): los proveedores, los listados de regalos y los productos de papelería (Growth Market Reports, 2025).

2.3. Herramientas existentes

A continuación se analizan tres de las plataformas más representativas: **Bodas.net**, referente en el mercado español, y **The Knot** y **Zola**, dos de las plataformas líderes en el mercado internacional.

2.3.1. Bodas.net

Bodas.net es una de las plataformas de referencia en España, con un punto fuerte en su extenso directorio de proveedores clasificados por categoría y localización. Entre sus herramientas gratuitas de organización incluye un **gestor de invitados** que permite agruparlos (familiares, amigos, compañeros de trabajo) y controlar su asistencia; un **organizador de mesas** de tipo «arrastrar y soltar», vinculado a la lista de invitados, con distintos tipos de mesa; un **presupuestador** que realiza una estimación automática de gasto por categoría, editable por el usuario; una **agenda de tareas** organizada por meses a modo de cuenta atrás; y la creación de una **web de boda** personalizada para compartir la información con los invitados. Toda la información se sincroniza entre la versión web y la aplicación móvil (Bodas.net, s.f.).

2.3.2. The Knot

The Knot es una de las plataformas líderes en Estados Unidos. Ofrece una **lista de invitados** con recogida de direcciones y seguimiento de confirmaciones (RSVP) por evento, una herramienta de **plano de mesas**, un **asesor de presupuesto** (*Budget Advisor*) que estima el coste medio de la boda y su reparto por proveedor a partir de datos reales de otras parejas en cada localización, una **lista de tareas** personalizable según la fecha, y una **web de boda** gratuita que sirve también para recoger las confirmaciones. En cuanto a inteligencia artificial, The Knot incorpora ayuda para recomendar proveedores (fotógrafos, floristas o espacios) que se ajusten al presupuesto y al estilo indicados por la pareja (The Knot, s.f.).

2.3.3. Zola

Zola se presenta como una solución «todo en uno» que reúne web, listado de regalos, lista de invitados y presupuesto en una misma aplicación. Su **lista de tareas** se reorganiza automáticamente en función de la fecha de la boda; su **gestor de invitados** recoge direcciones, confirmaciones, elección de menú e incluso peticiones musicales; su **plano de mesas** funciona por «arrastrar y soltar» y permite exportar el resultado a PDF; y su **seguimiento de presupuesto** ofrece sugerencias de reparto y recordatorios de pago. Zola incorpora además una función de IA para redactar mensajes de agradecimiento personalizados. Su servicio básico es gratuito, si bien algunas herramientas avanzadas, como el plano de mesas, tienen un coste adicional (Zola, s.f.).

2.3.4. Comparativa

La Tabla 2.1 resume las funcionalidades de las tres plataformas frente a la aplicación desarrollada en este proyecto.

Tabla 2.1. Comparativa de funcionalidades entre las plataformas existentes y este proyecto.

Funcionalidad	Bodas.net	The Knot	Zola	Este proyecto
Invitados y RSVP	Sí	Sí	Sí	Sí (con acompañantes)
Grupos de invitados	Sí	Sí	Sí	Sí
Distribución de mesas	Sí	Sí	Sí (de pago)	Sí
Tareas / checklist	Sí	Sí	Sí	Sí
Presupuesto	Sí	Sí	Sí	Sí
Proveedores	Sí (directorio)	Sí (directorio)	Sí (directorio)	Sí (gestión propia + documentos)
RSVP público / web	Sí	Sí	Sí	Sí (enlace público)
IA	Limitada	Recomendación de proveedores	Redacción de textos	Propuestas de tareas, mesas y presupuesto
Modelo	Gratuito (marketplace)	Gratuito (marketplace)	Gratuito (marketplace)	No comercial (TFM)

2.4. Limitaciones y oportunidad

Del análisis anterior se desprenden varias observaciones. En primer lugar, las plataformas existentes están construidas alrededor de un **modelo de negocio basado en el directorio de proveedores y el listado de regalos**: las herramientas de organización, aun siendo útiles y completas, son un complemento de ese modelo, y algunas prestaciones concretas —como el plano de mesas de Zola— pueden requerir un pago adicional. En segundo lugar, la **inteligencia artificial** que estas plataformas han empezado a incorporar se orienta principalmente a su negocio (recomendación de proveedores) o a tareas de comunicación (redacción de mensajes), más que a **automatizar las decisiones organizativas internas** de la pareja.

Existe, por tanto, la oportunidad de plantear una aplicación **integrada y autocontenida**, sin la capa de directorio comercial, en la que la inteligencia artificial se aplique precisamente a las decisiones más tediosas de la organización —**generar el listado de tareas, distribuir a los invitados en las mesas y repartir el presupuesto por categorías**— y lo haga en forma de **propuestas editables que la pareja revisa y confirma**. Esta es la propuesta de valor que persigue este proyecto y que lo diferencia de las soluciones analizadas.

2.5. Tecnologías y herramientas seleccionadas

Una vez situado el problema, se describen las tecnologías elegidas para la implementación. La aplicación se estructura en dos partes —cliente y servidor— desarrolladas ambas con **TypeScript**, un superconjunto de JavaScript que añade tipado estático y reduce los errores en tiempo de desarrollo (Microsoft, s.f.).

Cliente. La interfaz se ha construido con **React** (Meta Open Source, s.f.), una biblioteca para construir interfaces de usuario basada en componentes reutilizables. Frente a alternativas como Angular —un framework más completo pero también más pesado y prescriptivo— o Vue, se optó por React por su modelo de componentes sencillo, su enorme ecosistema y su amplia adopción, lo que facilita encontrar documentación y bibliotecas de apoyo. Como herramienta de construcción se eligió **Vite** (Vite, s.f.) en lugar de empaquetadores tradicionales como Webpack o de Create React App: Vite ofrece un servidor de desarrollo de arranque casi instantáneo y una recarga en caliente muy rápida, al apoyarse en los módulos nativos de JavaScript del navegador durante el desarrollo y en un empaquetado optimizado solo para la compilación de producción. Para los estilos se utilizó **Tailwind CSS** (Tailwind Labs, s.f.), un framework de utilidades que, a diferencia de bibliotecas de componentes cerradas como Bootstrap, aplica clases atómicas directamente en el marcado y evita escribir CSS a medida, lo que da consistencia visual y rapidez. Sobre él se empleó **shadcn/ui** (shadcn, s.f.), una colección de componentes accesibles contruidos sobre Radix UI que se integran en el propio proyecto y se pueden personalizar, en lugar de depender de una biblioteca externa con una estética impuesta. Para las gráficas y el calendario se emplearon bibliotecas especializadas.

Servidor. Se ha empleado **Node.js** (OpenJS Foundation, s.f.-b) con el framework **Express** (OpenJS Foundation, s.f.-a), que facilita la creación de una API REST (interfaz de programación de aplicaciones). Se prefirió Express frente a alternativas como NestJS —más estructurado, pero con un andamiaje y una curva de aprendizaje mayores— o Fastify, por su madurez, su simplicidad y su extenso catálogo de *middlewares*, adecuados para un proyecto de este tamaño. La validación de las entradas se realiza con **Zod** (Zod, s.f.), una biblioteca que define esquemas de validación en tiempo de ejecución y, a la vez, **inferire de ellos los tipos de TypeScript**, de modo que una única definición sirve para validar y tipar; frente a alternativas como Joi o class-validator, Zod destaca por esa integración con el sistema de tipos. El acceso a datos se realiza mediante el ORM (mapeador objeto-relacional) **Prisma** (Prisma, s.f.), que a partir de un esquema declarativo genera un cliente con tipado estático y gestiona las migraciones de la base de datos; frente a ORM basados en decoradores como TypeORM o Sequelize, Prisma mantiene una única fuente de verdad entre el esquema, los tipos y la base de datos. Como sistema gestor se eligió **PostgreSQL** (PostgreSQL Global Development Group, s.f.), un sistema relacional de código abierto ampliamente contrastado, preferido frente a MySQL por su rigor en la integridad referencial y su soporte de tipos avanzados (como los arrays empleados en el modelo de invitados). El entorno de base de datos se ejecuta en un contenedor **Docker** (Docker Inc., s.f.) para simplificar su puesta en marcha y reproducibilidad.

Por último, las asistencias de inteligencia artificial se apoyan en la **API de OpenAI** (OpenAI, s.f.), invocada exclusivamente desde el servidor. La Tabla 2.2 resume las principales decisiones tecnológicas, las alternativas consideradas y el motivo de cada elección. La justificación detallada de estas decisiones y su uso concreto se desarrollan en los capítulos de diseño (capítulo 7) e implementación (capítulo 8).

Tabla 2.2. Principales decisiones tecnológicas y alternativas consideradas.

Ámbito	Elección	Alternativas consideradas	Motivo principal
Lenguaje	TypeScript	JavaScript	Tipado estático; menos errores en desarrollo
Interfaz	React	Angular, Vue	Modelo de componentes y ecosistema amplio
Construcción	Vite	Webpack, Create React App	Arranque y recarga en caliente muy rápidos
Estilos	Tailwind CSS + shadcn/ui	Bootstrap, Material UI	Utilidades atómicas y componentes accesibles y personalizables
Servidor	Node.js + Express	NestJS, Fastify	Madurez, simplicidad y ecosistema de <i>middlewares</i>
Validación	Zod	Joi, class-validator	Validación e inferencia de tipos desde un único esquema
Acceso a datos	Prisma	TypeORM, Sequelize	Esquema declarativo y cliente con tipado estático
Base de datos	PostgreSQL	MySQL	Integridad referencial y tipos avanzados

3. OBJETIVOS

Este capítulo establece los objetivos del Trabajo Fin de Máster. Se distingue entre un **objetivo general**, que resume la meta global del proyecto, y un conjunto de **objetivos específicos**, más concretos y verificables, que se traducen posteriormente en los requisitos del sistema (capítulo 6) y cuyo grado de cumplimiento se evalúa en los resultados (capítulo 10).

3.1. Objetivo general

El objetivo general del proyecto es **desarrollar una aplicación web que permita a una pareja organizar de forma integral y centralizada todos los aspectos de su boda** — invitados, grupos, mesas, tareas, agenda, presupuesto y proveedores— desde un único panel, incorporando además un conjunto de **asistencias basadas en inteligencia artificial** que automaticen las decisiones organizativas más tediosas y actúen como elemento diferenciador frente a las soluciones existentes.

3.2. Objetivos específicos

Para alcanzar el objetivo general se definen los siguientes objetivos específicos:

- **OE-1. Gestión de la cuenta y de las bodas.** Proporcionar un sistema de registro e inicio de sesión seguro que permita a cada usuario gestionar una o varias bodas de forma aislada.
- **OE-2. Gestión de invitados.** Permitir el registro completo de invitados y sus acompañantes, con seguimiento de la asistencia (RSVP), el envío de invitaciones y una vía para que los invitados confirmen su asistencia.
- **OE-3. Organización de grupos y mesas.** Ofrecer la agrupación de invitados y la asignación de estos a las mesas y asientos del banquete de forma visual.
- **OE-4. Planificación de tareas y agenda.** Facilitar la gestión de las tareas pendientes y de los eventos del calendario asociados a la boda.
- **OE-5. Gestión económica.** Permitir el control del presupuesto y de los proveedores, y la relación entre ambos.
- **OE-6. Seguimiento global.** Proporcionar un panel que resuma el estado de la boda y un sistema de avisos sobre tareas y pagos próximos o vencidos.
- **OE-7. Asistencias con inteligencia artificial.** Integrar asistencias que propongan automáticamente un listado de tareas, una distribución de mesas y un reparto de presupuesto, siempre bajo la revisión y confirmación del usuario.
- **OE-8. Calidad del producto.** Construir la aplicación atendiendo a la seguridad, la usabilidad (interfaz responsive y localizada), la robustez frente a errores y la calidad del código (pruebas e integración continua).

3.3. Relación de los objetivos con los requisitos

La Tabla 3.1 relaciona cada objetivo específico con los requisitos que lo materializan (definidos en el capítulo 6). Esta correspondencia permite comprobar, en el capítulo de

resultados, en qué medida se ha alcanzado cada objetivo.

Tabla 3.1. Correspondencia entre objetivos específicos y requisitos.

Objetivo	Requisitos asociados
OE-1. Cuenta y bodas	RF-01...RF-09, RNF-01, RNF-02
OE-2. Invitados	RF-10...RF-16, RF-80, RF-81
OE-3. Grupos y mesas	RF-20...RF-24
OE-4. Tareas y agenda	RF-30, RF-31, RF-40, RF-41
OE-5. Gestión económica	RF-50...RF-52, RF-60...RF-62
OE-6. Seguimiento global	RF-70, RF-71
OE-7. Asistencias con IA	IA-01, IA-02, IA-03, IA-90, IA-91, IA-92
OE-8. Calidad	RNF-03, RNF-10...RNF-13, RNF-20...RNF-23

4. METODOLOGÍA

En este capítulo se describe la forma en que se ha organizado el desarrollo del proyecto: la metodología de trabajo empleada y su justificación, las herramientas y el entorno utilizados, la organización del trabajo por iteraciones y, por último, los principales problemas encontrados durante el desarrollo y cómo se resolvieron.

4.1. Metodología de desarrollo

El proyecto se ha desarrollado siguiendo un enfoque **iterativo e incremental**. En lugar de intentar construir la aplicación completa de una sola vez, se partió de un **núcleo mínimo funcional** y, sobre él, se fueron **añadiendo y refinando módulos** de forma sucesiva: cada nueva funcionalidad se integraba en una versión ya operativa, se probaba y servía de base para la siguiente.

Esta elección responde a las características del proyecto. Al tratarse de un trabajo realizado por **una sola persona** y cuyo alcance se fue **concretando y ampliando** a medida que avanzaba —añadiendo prestaciones a cada apartado según se identificaban nuevas necesidades—, un modelo rígido y secuencial habría resultado poco adecuado. Un modelo en cascada exige tener todos los requisitos cerrados desde el inicio; aquí, en cambio, la posibilidad de disponer pronto de una versión utilizable y de **incorporar mejoras de forma progresiva** encajaba mucho mejor con la realidad del desarrollo. Cada incremento siguió internamente el mismo ciclo —análisis, diseño, implementación y prueba— aplicado a un alcance acotado, tal como muestra la Figura 4.1.

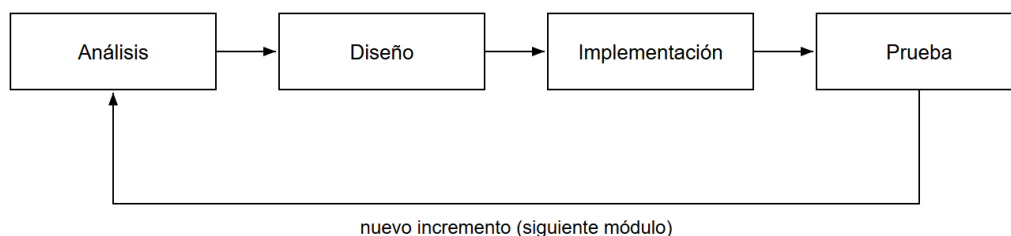


Figura 4.1. Ciclo de desarrollo iterativo e incremental aplicado a cada módulo. *Fuente: elaboración propia.*

4.2. Herramientas y entorno de trabajo

El desarrollo se apoyó en las siguientes herramientas:

- **Editor de código.** Se utilizó Visual Studio Code como entorno de desarrollo, por su integración con el control de versiones y su soporte para JavaScript/TypeScript.
- **Control de versiones.** El proyecto se gestionó con Git y un repositorio remoto, lo que permitió mantener un historial de cambios y recuperar versiones anteriores.
- **Base de datos.** Se ejecutó PostgreSQL en un contenedor mediante Docker, lo que simplifica la puesta en marcha del entorno local y su reproducibilidad. Para inspeccionar y editar los datos durante el desarrollo se empleó la herramienta visual del ORM.

- **Pruebas e integración continua.** Durante el desarrollo se verificaron manualmente los flujos de cada módulo y se escribió un conjunto de **pruebas automatizadas** en el backend. Además, se configuró un proceso de **integración continua** que, ante cada cambio, valida la compilación, el estilo y las pruebas, evitando que se introduzcan regresiones (capítulo 9).

4.3. Organización del trabajo por iteraciones

El trabajo se organizó en iteraciones sucesivas, cada una centrada en un conjunto coherente de funcionalidades. De forma resumida:

1. **Núcleo.** Autenticación de usuarios (registro, inicio y cierre de sesión) y gestión de invitados, que constituyen la base sobre la que se apoya el resto del sistema.
2. **Módulos de organización.** Grupos, mesas y asignación de asientos, tareas y agenda.
3. **Módulos económicos y de seguimiento.** Presupuesto, proveedores y el panel de seguimiento que agrega el estado de la boda.
4. **Funcionalidades avanzadas y endurecimiento.** Envío de invitaciones y confirmación pública de asistencia, asistencias con inteligencia artificial, y las mejoras transversales de seguridad, pruebas y calidad.

Dentro de cada iteración, la implementación de un módulo seguía un orden natural derivado de la arquitectura: primero se definía su representación en el **modelo de datos**, después la lógica del **servidor** (servicio, validación y ruta) y, por último, la **interfaz** que lo consume, cerrando con una prueba manual del flujo completo.

4.4. Problemas encontrados durante el desarrollo

Como en cualquier proyecto de software, durante el desarrollo surgieron dificultades que obligaron a replantear soluciones. Se destacan cuatro por su relevancia.

4.4.1. La representación visual de las mesas

Uno de los aspectos que más iteración requirió no fue la lógica de datos, sino **cómo presentar las mesas de forma visual y comprensible**. La asignación de invitados a asientos es intuitiva sobre el papel, pero trasladarla a una interfaz clara —que mostrara las mesas, sus asientos libres y ocupados, y permitiera asignar a una persona a un sitio concreto— no era evidente. Se probaron distintas formas de disposición hasta llegar a un **mapa de mesas** en el que cada mesa muestra sus asientos y el usuario coloca al invitado seleccionado en el asiento deseado, con acciones para liberar un asiento o vaciar una mesa.

4.4.2. El aislamiento entre bodas

Al permitir que un usuario gestione varias bodas y que existan varios usuarios, se hizo evidente la necesidad de **impedir que nadie accediera a datos de una boda que no es suya**. Un primer enfoque, en el que cada consulta se limitaba a la boda indicada por el cliente, resultaba insuficiente: manipulando el identificador de un recurso se podía intentar acceder a datos ajenos. La solución consistió en establecer una **doble barrera**

de autorización: un control previo que comprueba que la boda pertenece al usuario y, además, la restricción de todas las consultas por propietario. Esta decisión, descrita en el capítulo 7, se convirtió en un principio transversal del sistema.

4.4.3. La expiración de la sesión

La autenticación basada en un único *token* de vida corta presentaba un inconveniente de usabilidad: cuando el *token* caducaba, el usuario era **expulsado al inicio de sesión** en mitad de su trabajo. Resolverlo obligó a introducir un segundo *token* de larga duración con el que **renovar la sesión de forma transparente**, y a llevar un registro de las sesiones activas para poder invalidarlas de verdad al cerrar sesión. El resultado es que la sesión se mantiene con naturalidad mientras es legítima, sin sacrificar la seguridad.

4.4.4. La codificación y el separador en la importación CSV

La importación de invitados desde un fichero CSV reveló un problema al abrir dichos ficheros en la versión española de Excel: **los acentos y la «ñ» se corrompían y las columnas no se separaban correctamente**. La causa estaba en la codificación y en el separador de columnas que Excel asume según la configuración regional. La solución — anteponer la marca de codificación UTF-8 y usar el punto y coma como separador— permitió que los ficheros generados por la aplicación se abrieran correctamente sin que el usuario tuviera que ajustar nada, un detalle pequeño pero importante para la experiencia de uso.

Estos problemas, y otros de menor entidad, se tradujeron en aprendizajes que se recogen en las conclusiones (capítulo 11).

5. PLANIFICACIÓN Y ESTIMACIÓN DE COSTES

En este capítulo se describe la planificación temporal del proyecto y se realiza una estimación de los costes asociados a su desarrollo. La planificación se apoya en la metodología iterativa e incremental descrita en el capítulo 4, y la estimación de costes distingue entre costes de personal, de software y de hardware.

5.1. Planificación temporal

El desarrollo se organizó en las iteraciones descritas en el capítulo anterior, a las que se añaden una fase inicial de análisis y diseño, una fase de pruebas y validación y la redacción de la memoria, que se desarrolló de forma solapada con las últimas iteraciones. La Figura 5.1 presenta el diagrama de Gantt con la distribución temporal aproximada de estas tareas a lo largo del curso académico.

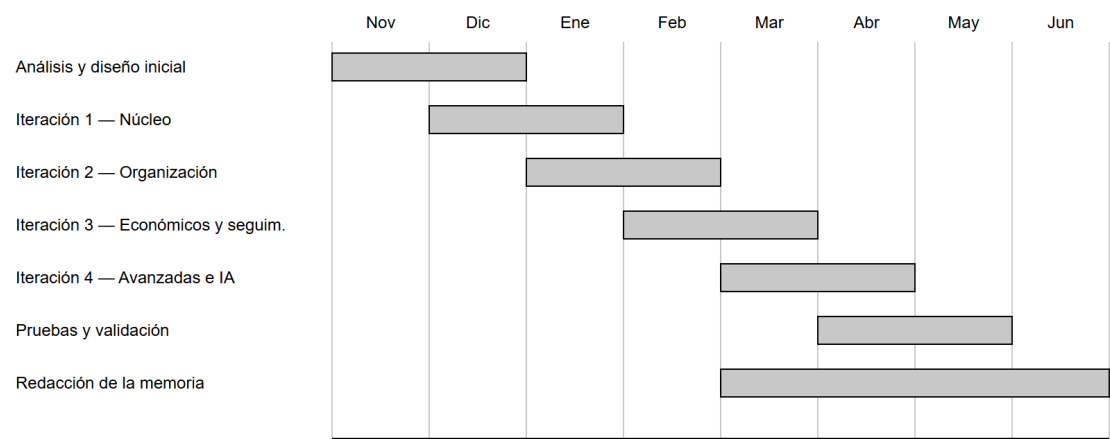


Figura 5.1. Planificación temporal del proyecto (diagrama de Gantt). *Fuente: elaboración propia.*

Como se aprecia en la figura, las iteraciones se encadenan de forma que cada una comienza antes de que finalice completamente la anterior, lo que refleja el carácter incremental del desarrollo. Las pruebas y la redacción de la memoria se extienden sobre las fases finales, en paralelo al último trabajo de implementación.

La Tabla 5.1 detalla las tareas en que se descompuso el proyecto, con su semana de inicio y de fin —tomando como referencia la primera semana de noviembre como semana 1— y su duración aproximada. El solapamiento entre tareas refleja el carácter incremental del desarrollo.

Tabla 5.1. Planificación de tareas por semanas.

Tarea	Semana inicio	Semana fin	Duración
Análisis y diseño inicial	S1	S4	4 semanas
Iteración 1 — Núcleo (autenticación y gestión de invitados)	S3	S9	7 semanas
Iteración 2 — Organización (grupos, mesas, tareas y agenda)	S8	S15	8 semanas
Iteración 3 — Económicos y seguimiento (presupuesto, proveedores y panel)	S14	S21	8 semanas
Iteración 4 — Avanzadas e IA (invitaciones/RSVP, asistencias de IA y endurecimiento)	S20	S28	9 semanas
Pruebas y validación	S26	S32	7 semanas
Redacción de la memoria	S22	S34	13 semanas

Cada iteración se descompone internamente en las tareas de análisis, diseño, implementación y prueba de los módulos que la componen, siguiendo el ciclo descrito en el capítulo 4. La duración total del proyecto abarca aproximadamente treinta y cuatro semanas del curso académico, entre noviembre y junio.

5.2. Estimación de costes

La estimación de costes se descompone en tres partidas: personal, software y hardware. Al tratarse de un proyecto de fin de máster desarrollado por una sola persona, la partida más relevante es la de personal.

5.2.1. Costes de personal

Se estima una dedicación aproximada de 600 horas de desarrollo, correspondientes a las distintas iteraciones y a las fases de análisis, pruebas y documentación, a las que se añade una dedicación menor en concepto de dirección y tutorización. La Tabla 5.2 recoge la estimación, empleando tarifas de referencia del mercado.

Tabla 5.2. Estimación de costes de personal.

Rol	Horas	Coste/hora	Total
Ingeniero de software (desarrollo)	600 h	20 €/h	12.000 €
Dirección y tutorización	25 h	45 €/h	1.125 €
Total			13.125 €

5.2.2. Costes de software

Todas las herramientas de desarrollo empleadas son de código abierto, bajo licencias permisivas (MIT, Apache 2.0, etc.), o servicios con un plan gratuito, por lo que su coste de adquisición es nulo. El único coste reseñable corresponde al uso de la API de inteligencia artificial, facturada por consumo. La Tabla 5.3 detalla cada herramienta, su licencia de uso y su coste.

Tabla 5.3. Herramientas de software, licencia y coste.

Herramienta	Licencia	Coste
Node.js, React, Express, Vite, Tailwind CSS, Zod, Vitest	MIT	0 €
Prisma, Docker Engine	Apache 2.0	0 €
PostgreSQL	PostgreSQL License	0 €
Git	GPL v2	0 €
Visual Studio Code	MIT	0 €
GitHub (repositorio remoto)	SaaS, plan gratuito	0 €
API de OpenAI (modelo gpt-4o-mini)	Comercial, pago por consumo	5 €
Total		5 €

El coste de la **API de OpenAI** no corresponde a una suscripción de tarifa plana, sino a un modelo de **pago por consumo** en función del número de *tokens* procesados. Con el modelo empleado, `gpt-4o-mini`, el precio de referencia es de aproximadamente 0,15 \$ por millón de *tokens* de entrada y 0,60 \$ por millón de *tokens* de salida. Durante el desarrollo se realizaron numerosas pruebas de las tres asistencias (tareas, mesas y presupuesto), con un consumo estimado del orden de varios millones de *tokens*, lo que se traduce en un coste aproximado de 5 €.

5.2.3. Costes de hardware

El desarrollo se realizó sobre un único ordenador portátil. Su coste se imputa de forma proporcional al tiempo de uso durante el proyecto, considerando una vida útil de cuatro años. La Tabla 5.4 recoge esta estimación.

Tabla 5.4. Estimación de costes de hardware.

Hardware	Coste	Amortización	Coste imputado
Ordenador portátil de desarrollo	1.000 €	8 meses de uso sobre 48 de vida útil	167 €
Total			167 €

5.2.4. Coste total

Sumando las tres partidas anteriores, la Tabla 5.5 presenta el coste total estimado del proyecto.

Tabla 5.5. Coste total estimado del proyecto.

Partida	Coste
Personal	13.125 €
Software	5 €
Hardware	167 €
Total	13.297 €

El coste está dominado por la partida de personal, como es habitual en un proyecto de desarrollo software, mientras que el uso de tecnologías de código abierto mantiene los costes de software y hardware en valores reducidos.

6. ANÁLISIS

En este capítulo se especifica el comportamiento esperado de la aplicación antes de entrar en su diseño e implementación. Se parte de los objetivos definidos en el capítulo 3 y se traducen en un catálogo de **requisitos funcionales** (qué debe permitir hacer el sistema) y **requisitos no funcionales** (cómo debe hacerlo: seguridad, usabilidad, rendimiento o calidad). A continuación se identifican los **actores** que interactúan con la aplicación y se describen los **casos de uso** más representativos. El capítulo se cierra con una **matriz de trazabilidad** que relaciona cada requisito con los casos de uso que lo ejercitan y con su estado de implementación, de modo que la cobertura del sistema pueda comprobarse de forma objetiva.

Los requisitos se han capturado de forma **incremental**: se partió de un núcleo mínimo — autenticación y gestión de invitados— y, a medida que la aplicación crecía, se fueron incorporando y refinando los requisitos de cada nuevo módulo (mesas, tareas, agenda, presupuesto, proveedores) hasta completar el producto. Cada requisito se identifica con un código de la forma **RF-nn** (funcional), **RNF-nn** (no funcional) o **IA-nn** (funcional específico de las asistencias con inteligencia artificial), agrupados por dominio. Esta codificación se utiliza como referencia única en el resto de la memoria.

6.1. Requisitos funcionales

Los requisitos funcionales describen los servicios que la aplicación ofrece a sus usuarios. Se presentan agrupados por módulo. La columna **Prioridad** distingue entre requisitos imprescindibles para un producto mínimo viable (**Alta**), deseables (**Media**) y complementarios (**Baja**).

6.1.1. Autenticación y gestión de la cuenta

Código	Descripción	Prioridad
RF-01	El usuario podrá registrarse indicando nombre, correo y contraseña. Al registrarse se crea automáticamente su primera boda.	Alta
RF-02	El usuario podrá iniciar sesión con correo y contraseña y obtener acceso a su boda activa.	Alta
RF-03	El sistema protegerá todas las funcionalidades privadas exigiendo una sesión válida, redirigiendo al inicio de sesión cuando la sesión no exista o haya expirado.	Alta
RF-04	El sistema mantendrá la sesión activa de forma segura, renovándola sin necesidad de que el usuario vuelva a introducir sus credenciales mientras la sesión siga siendo válida.	Media
RF-05	El usuario podrá cerrar sesión, invalidando la sesión en el servidor.	Media
RF-06	El usuario podrá recuperar su contraseña mediante un enlace enviado por correo, y cambiarla estando autenticado.	Media
RF-07	El usuario podrá consultar y editar los datos de su perfil (nombre y correo).	Baja

6.1.2. Gestión de bodas

Código	Descripción	Prioridad
RF-08	El usuario podrá gestionar varias bodas y seleccionar cuál es la boda activa, sobre la que operan el resto de módulos.	Baja
RF-09	El usuario podrá editar el nombre y la fecha de la boda; la fecha alimenta la cuenta atrás mostrada en la interfaz.	Alta

6.1.3. Gestión de invitados

Código	Descripción	Prioridad
RF-10	El usuario podrá crear, consultar, editar y eliminar invitados.	Alta
RF-11	El usuario podrá asociar acompañantes a un invitado principal, gestionándolos en la misma operación.	Alta
RF-12	Cada invitado registrará su estado de asistencia (pendiente/confirmado/rechazado), tipo de dieta, alergias, grupo de edad, teléfono, correo y notas.	Alta
RF-13	El usuario podrá importar invitados de forma masiva desde un fichero CSV (valores separados por comas), creando el grupo indicado si no existe.	Media
RF-14	El sistema registrará qué invitaciones se han enviado y cuándo.	Media
RF-15	El usuario podrá eliminar varios invitados seleccionados en una sola operación.	Baja
RF-16	El usuario podrá exportar la lista de invitados a CSV.	Baja

6.1.4. Grupos y mesas

Código	Descripción	Prioridad
RF-20	El usuario podrá crear, editar y eliminar grupos de invitados.	Alta
RF-21	El usuario podrá asignar invitados a un grupo, también de forma masiva desde la vista de grupos.	Media
RF-22	El usuario podrá crear mesas indicando su número de asientos.	Alta
RF-23	El usuario podrá asignar manualmente invitados a un asiento concreto de una mesa, con un mapa visual de la distribución, garantizando que un asiento no se asigne dos veces.	Alta
RF-24	El usuario podrá vaciar una mesa o liberar un asiento concreto.	Media

6.1.5. Tareas

Código	Descripción	Prioridad
RF-30	El usuario podrá crear, editar y eliminar tareas con título, notas y fecha límite.	Alta
RF-31	Cada tarea tendrá prioridad, estado y categoría del dominio de la boda, y podrá filtrarse por ellos.	Alta

6.1.6. Agenda

Código	Descripción	Prioridad
RF-40	El usuario podrá crear, editar y eliminar eventos con fecha, hora, ubicación y descripción.	Alta
RF-41	El sistema mostrará los eventos en una vista de calendario y una cuenta atrás hasta la fecha de la boda.	Media

6.1.7. Presupuesto

Código	Descripción	Prioridad
RF-50	El usuario podrá definir un presupuesto global para la boda (importe y moneda).	Alta
RF-51	El usuario podrá gestionar partidas de gasto con importe estimado, importe real, importe pagado, estado, fechas, categoría y proveedor asociado.	Alta
RF-52	El sistema mostrará gráficas de reparto y seguimiento del presupuesto.	Media

6.1.8. Proveedores

Código	Descripción	Prioridad
RF-60	El usuario podrá gestionar proveedores con categoría, estado de contratación, datos de contacto, precios estimado y final, web y notas.	Alta
RF-61	El usuario podrá vincular un proveedor a una partida de presupuesto.	Media
RF-62	El usuario podrá adjuntar, listar, descargar y eliminar documentos (contratos, presupuestos) de cada proveedor.	Media

6.1.9. Panel de seguimiento y avisos

Código	Descripción	Prioridad
RF-70	El sistema ofrecerá un panel con el resumen agregado de la boda: invitados por asistencia y edad, ocupación de mesas, progreso de tareas, estado del presupuesto, invitaciones enviadas, proveedores y próximos eventos.	Alta
RF-71	El sistema avisará in-app de las tareas no completadas y los pagos pendientes vencidos o próximos a vencer.	Baja

6.1.10. Invitaciones y confirmación de asistencia (RSVP)

Código	Descripción	Prioridad
RF-80	El sistema enviará invitaciones por correo electrónico a los invitados con dirección de correo, registrando las enviadas con éxito.	Alta
RF-81	Cada invitado dispondrá de un enlace público único mediante el cual podrá confirmar su asistencia, dieta, alergias y la de sus acompañantes, sin necesidad de iniciar sesión.	Media

6.1.11. Asistencias con inteligencia artificial

Las funciones de IA comparten un principio común: reciben el contexto de la boda desde la base de datos, devuelven **propuestas editables** que el usuario revisa y confirma, y **nunca escriben en la base de datos sin confirmación explícita**.

Código	Descripción	Prioridad
IA-01	El sistema podrá generar una propuesta de tareas/checklist adaptada a la boda (fecha, número de invitados y tareas ya existentes), que el usuario revisa y confirma antes de crearlas.	Media
IA-02	El sistema podrá proponer una distribución de invitados en mesas a partir de las mesas y los grupos existentes, validando la propuesta antes de aplicarla.	Media
IA-03	El sistema podrá proponer un reparto de presupuesto por categorías y una lista de proveedores acorde con el presupuesto total y el número de invitados.	Media

6.2. Requisitos no funcionales

Los requisitos no funcionales establecen las cualidades del sistema. Se organizan en seguridad, calidad de código y requisitos generales.

Código	Categoría	Descripción	Prioridad
RNF-01	Seguridad	Cada usuario solo podrá acceder a los datos de sus propias bodas; toda operación con alcance de boda verificará la propiedad, devolviendo «no encontrado» ante accesos ajenos.	Alta
RNF-02	Seguridad	Las contraseñas se almacenarán cifradas (hash). El servidor validará al arrancar que los secretos de firma de sesión estén correctamente configurados en producción.	Alta
RNF-03	Seguridad	El acceso a los endpoints de autenticación estará limitado por frecuencia (rate limiting) y el origen de las peticiones restringido por entorno (CORS).	Baja
RNF-10	Calidad	El código seguirá una arquitectura en capas por dominio y reutilizará una única conexión a la base de datos.	Media
RNF-11	Calidad	La configuración por entorno estará documentada y externalizada en variables de entorno.	Media
RNF-12	Calidad	El backend dispondrá de pruebas automatizadas (unitarias y de integración).	Baja
RNF-13	Calidad	El proyecto contará con integración continua que valide compilación, estilo y pruebas en cada cambio.	Baja
RNF-20	Usabilidad	La interfaz será responsive y accesible, con navegación adaptada a móvil y textos alternativos en los controles.	Alta
RNF-21	Localización	La aplicación estará en español, con formatos de fecha y moneda es-ES/EUR.	Alta
RNF-22	Robustez	El sistema tratará los errores de forma uniforme, devolviendo mensajes claros al usuario sin filtrar detalles internos.	Alta
RNF-23	Usabilidad	El sistema confirmará cada acción relevante mediante avisos (toasts) de éxito o error y solicitará confirmación antes de las operaciones destructivas.	Media
IA-90	Seguridad	La clave de la API de IA residirá exclusivamente en el backend; el cliente nunca la verá.	Alta
IA-91	Robustez	Las respuestas de la IA se solicitarán con formato estructurado y se validarán antes de usarse.	Alta
IA-92	Robustez	Las funciones de IA degradarán con elegancia: aviso claro si no están configuradas y mensaje amable si la API falla.	Media

6.3. Actores

Un actor es toda entidad externa que interactúa con el sistema. En esta aplicación se identifican dos actores humanos y, de forma auxiliar, los servicios externos con los que el sistema se integra.

- **Usuario registrado (organizador de la boda).** Es el actor principal: la persona o pareja que organiza la boda. Tras autenticarse, gestiona la totalidad de los módulos (invitados, grupos, mesas, tareas, agenda, presupuesto y proveedores) sobre sus propias bodas. Es el único actor con cuenta y sesión.

- **Invitado.** Actor externo **sin cuenta** en el sistema. Interactúa únicamente a través del enlace público de confirmación de asistencia (RF-81), donde comunica su asistencia, dieta y alergias. No accede a ninguna otra parte de la aplicación.
- **Servicios externos (auxiliares).** El sistema se apoya en un servicio de correo (para el envío de invitaciones y recuperación de contraseña) y en la API de inteligencia artificial (para las asistencias IA-01/02/03). No son actores en sentido estricto, pero se documentan porque condicionan varios casos de uso.

No existe un rol de administrador ni distinción de permisos entre usuarios: el modelo de autorización se basa por completo en la **propiedad de la boda** (RNF-01), de modo que cada usuario es administrador de sus propias bodas y de nada más.

La Figura 6.1 muestra el diagrama general de casos de uso, con los dos actores identificados y sus principales interacciones con el sistema.

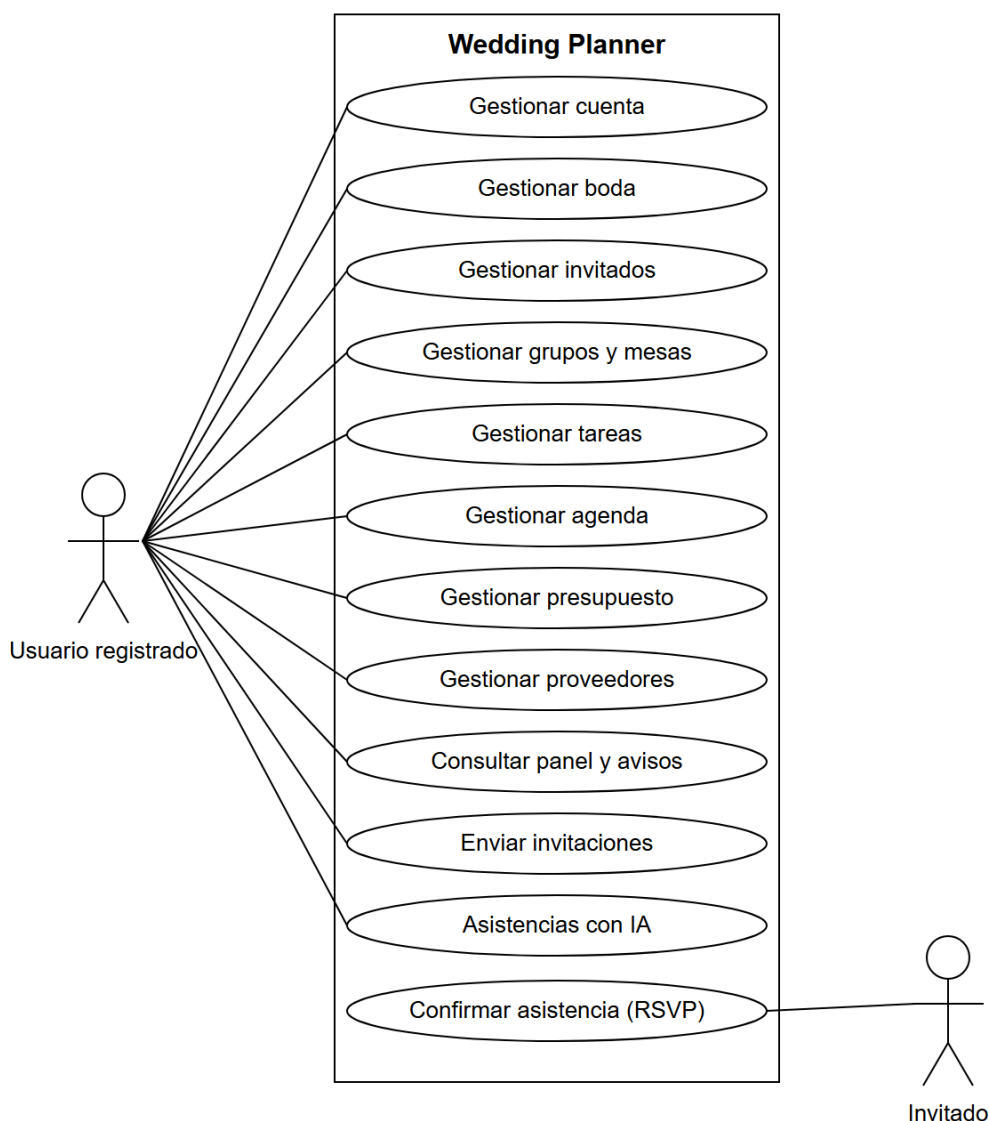


Figura 6.1. Diagrama de casos de uso general. *Fuente: elaboración propia.*

6.4. Casos de uso

Los casos de uso describen las interacciones concretas entre los actores y el sistema. Dado que la mayor parte de los módulos siguen un patrón de gestión (altas, bajas, modificaciones y consultas), no se documenta cada operación elemental como un caso de uso independiente; en su lugar se detallan los **procesos más representativos o con lógica no trivial**, que son los que aportan valor descriptivo. El resto de operaciones CRUD (crear, leer, actualizar y borrar) quedan recogidas de forma agregada en la matriz de trazabilidad (§6.5).

Se detallan cuatro casos de uso: alta e inicio de sesión, gestión de un invitado con confirmación pública de asistencia, distribución de mesas asistida por IA y seguimiento del presupuesto.

CU-01. Registro e inicio de sesión

Campo	Contenido
Código	CU-01
Descripción	Alta de un nuevo usuario e inicio de sesión en la aplicación.
Actor	Usuario registrado
Precondición	El usuario no dispone de sesión activa.
Postcondición	El usuario queda autenticado y con una boda activa creada.
Prioridad	Alta
Requisitos	RF-01, RF-02, RF-03, RF-04, RNF-02
Escenario principal	1. El usuario accede a la página de registro. 2. Introduce nombre, correo y contraseña. 3. El sistema valida los datos, crea la cuenta con la contraseña cifrada y una boda inicial. 4. El usuario inicia sesión con sus credenciales. 5. El sistema devuelve la sesión y redirige al panel de seguimiento.
Escenarios alternativos	3a. El correo ya existe → el sistema informa del conflicto y no crea la cuenta. 4a. Credenciales incorrectas → el sistema informa del error sin revelar cuál de los dos campos falla.

CU-02. Gestión de un invitado y confirmación pública de asistencia

Campo	Contenido
Código	CU-02
Descripción	Alta de un invitado con acompañantes, envío de invitación y confirmación de asistencia por parte del invitado sin iniciar sesión.
Actor	Usuario registrado (alta e invitación); Invitado (confirmación)
Precondición	El usuario tiene sesión activa y una boda seleccionada.
Postcondición	El invitado queda registrado y su asistencia actualizada según la respuesta recibida.
Prioridad	Alta
Requisitos	RF-10, RF-11, RF-12, RF-14, RF-80, RF-81, RNF-01
Escenario principal	1. El usuario da de alta un invitado con sus datos y acompañantes. 2. Solicita el envío de la invitación por correo. 3. El sistema envía el mensaje con el enlace único de confirmación y marca la invitación como enviada. 4. El invitado abre el enlace público. 5. Indica su asistencia, dieta y alergias, y las de sus acompañantes. 6. El sistema actualiza el estado de asistencia.
Escenarios alternativos	2a. El invitado no tiene correo → el sistema lo omite del envío y lo refleja en el resumen. 4a. El enlace no es válido → el sistema muestra un mensaje de error.

CU-03. Distribución de mesas asistida por IA

Campo	Contenido
Código	CU-03
Descripción	Generación de una propuesta de distribución de invitados en mesas mediante IA, revisión y aplicación.
Actor	Usuario registrado
Precondición	Existen mesas e invitados en la boda activa y la funcionalidad de IA está configurada.
Postcondición	Los invitados quedan asignados a mesas y asientos según la propuesta aceptada.
Prioridad	Media
Requisitos	RF-22, RF-23, IA-02, IA-90, IA-91, IA-92, RNF-01
Escenario principal	1. El usuario solicita una distribución con IA. 2. El sistema envía a la IA las mesas y los invitados y recibe una propuesta estructurada. 3. El sistema valida y depura la propuesta (descarta identificadores inexistentes, asientos fuera de capacidad y repeticiones). 4. Muestra la propuesta agrupada por mesa para su revisión. 5. El usuario la aplica. 6. El sistema reemplaza la distribución actual en una única transacción.
Escenarios alternativos	1a. La IA no está configurada → el sistema oculta la función e informa. 2a. La API falla o devuelve un formato inválido → el sistema muestra un mensaje amable sin aplicar cambios.

CU-04. Seguimiento del presupuesto

Campo	Contenido
Código	CU-04
Descripción	Definición del presupuesto, alta de partidas de gasto y seguimiento de pagos.
Actor	Usuario registrado
Precondición	El usuario tiene sesión activa y una boda seleccionada.
Postcondición	El presupuesto y sus partidas quedan actualizados y reflejados en las gráficas y avisos.
Prioridad	Alta
Requisitos	RF-50, RF-51, RF-52, RF-61, RF-71, RNF-01
Escenario principal	1. El usuario define el presupuesto total. 2. Añade partidas de gasto con su categoría, importe estimado y, opcionalmente, el proveedor vinculado. 3. Registra pagos y estados. 4. El sistema actualiza las gráficas de reparto y seguimiento. 5. Las partidas con pago vencido o próximo generan un aviso in-app.
Escenarios alternativos	2a. El proveedor vinculado no pertenece a la boda → el sistema rechaza la operación.

6.5. Matriz de trazabilidad

La siguiente matriz relaciona cada requisito con el caso de uso que lo ejercita y con su estado de implementación en el sistema entregado. Permite comprobar de un vistazo la cobertura del análisis y sirve de base para el capítulo de Resultados, donde se enlazará además con las pruebas realizadas. Los estados posibles son **Implementado**, **Parcial** y **Pendiente**.

Requisito	Caso de uso	Módulo	Estado
RF-01...RF-07	CU-01	Autenticación	Implementado
RF-08, RF-09	—	Gestión de bodas	Implementado
RF-10...RF-16	CU-02	Invitados	Implementado
RF-20, RF-21	CU-02	Grupos	Implementado
RF-22, RF-23, RF-24	CU-03	Mesas	Implementado
RF-30, RF-31	—	Tareas	Implementado
RF-40, RF-41	—	Agenda	Implementado
RF-50, RF-51, RF-52	CU-04	Presupuesto	Implementado
RF-60, RF-61, RF-62	CU-04	Proveedores	Implementado
RF-70, RF-71	CU-04	Panel y avisos	Implementado
RF-80, RF-81	CU-02	Invitaciones/RSVP	Implementado
IA-01	—	IA · Tareas	Implementado
IA-02	CU-03	IA · Mesas	Implementado
IA-03	—	IA · Presupuesto	Implementado
RNF-01...RNF-03	CU-01, CU-02, CU-03	Seguridad	Implementado
RNF-10...RNF-13	—	Calidad	Implementado ⁽¹⁾
RNF-20...RNF-23	—	Usabilidad/Robustez	Implementado
IA-90, IA-91, IA-92	CU-03	IA · Transversal	Implementado

⁽¹⁾ Las pruebas automatizadas cubren el backend; el frontend no dispone todavía de pruebas automatizadas (deuda técnica reconocida en el capítulo 9).

7. DISEÑO

Una vez definido *qué* debe hacer la aplicación (capítulo 6), este capítulo describe *cómo* se ha construido: la arquitectura general del sistema, el modelo de datos que sustenta la información, la forma en que el cliente y el servidor se comunican, los mecanismos de autenticación y autorización, la estructura de navegación y, por último, el diseño de la interfaz. El objetivo es dejar documentadas las decisiones de diseño y su justificación, de modo que la implementación (capítulo 8) sea la traducción directa de lo aquí planteado.

7.1. Arquitectura general

La aplicación sigue una **arquitectura cliente-servidor** de tres niveles: una interfaz de usuario que se ejecuta en el navegador, un servidor que expone una API y contiene la lógica de negocio, y una base de datos relacional que persiste la información. El código se organiza en un único repositorio (*monorepo*) con **dos paquetes independientes** — `frontend` y `backend` — que se despliegan y ejecutan por separado.

El **backend** se estructura en **capas por dominio**, con un módulo por cada entidad del sistema (invitados, grupos, mesas, tareas, etc.). Cada petición atraviesa tres capas bien diferenciadas:

- **Rutas** (`routes`): definen los puntos de acceso de la API y el verbo HTTP asociado.
- **Controladores** (`controllers`): son responsables de **toda la validación** de la entrada (parámetros, cuerpo y consulta) mediante esquemas, dan forma a la respuesta HTTP y delegan la lógica en la capa de servicio.
- **Servicios** (`services`): contienen la **lógica de negocio** y el acceso a la base de datos a través del ORM. Las operaciones que implican varias escrituras se resuelven en transacción.

Esta separación estricta —validación en el controlador, lógica en el servicio— favorece la mantenibilidad y la capacidad de prueba (RNF-10), ya que cada capa tiene una única responsabilidad y puede evolucionar o testearse de forma aislada.

El **frontend** es una **aplicación de página única** (SPA). Todas las llamadas de red se canalizan a través de una **única capa de acceso a la API**, que centraliza la inclusión del *token* de sesión, el control de expiración y el tratamiento de errores; sobre ella, cada dominio dispone de un servicio propio que encapsula sus operaciones y los tipos y etiquetas en español asociados.

La **persistencia** se resuelve con una base de datos **PostgreSQL** accedida mediante un **ORM** que, a partir de un esquema declarativo, genera un cliente con tipado estático y gestiona las migraciones. Por último, el servidor se integra con dos **servicios externos**: un servicio de correo (SMTP) para el envío de invitaciones y la recuperación de contraseña, y la API de inteligencia artificial para las asistencias descritas en el capítulo 6. Ambas integraciones residen **exclusivamente en el backend**; el cliente nunca maneja credenciales de estos servicios (IA-90).

La Figura 7.1 presenta la arquitectura general del sistema, con sus tres niveles y las integraciones externas.

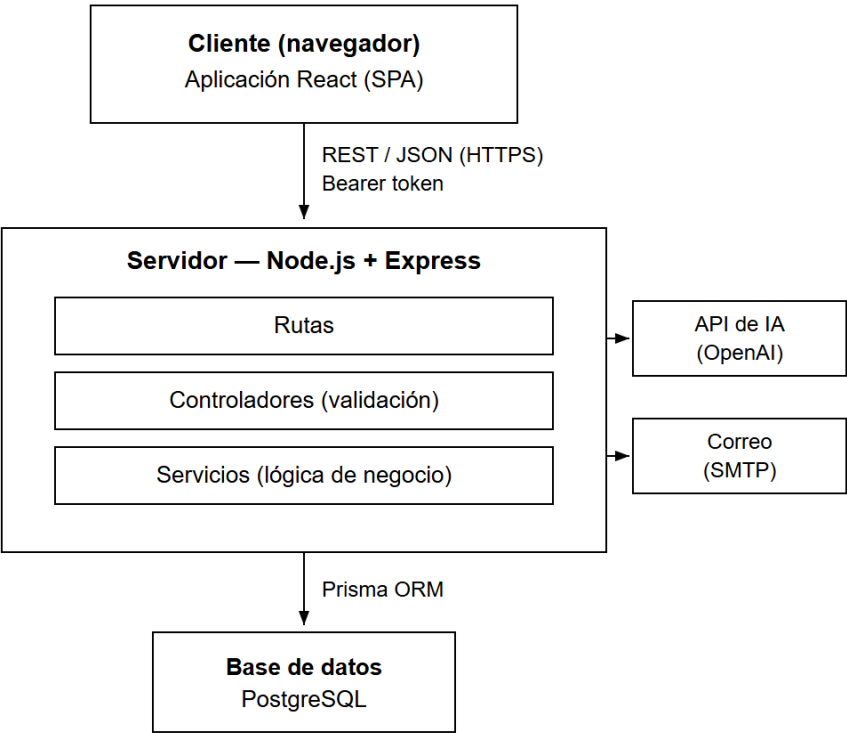


Figura 7.1. Arquitectura general de la aplicación. *Fuente: elaboración propia.*

El flujo por capas descrito puede visualizarse mediante un diagrama de clases. La Figura 7.2 lo ilustra para el módulo de invitados, representativo del resto de módulos: el controlador (GuestController) valida la entrada y delega en el servicio (GuestService), que contiene la lógica de negocio y gestiona la entidad de dominio (Guest). Todos los módulos del sistema siguen esta misma estructura.

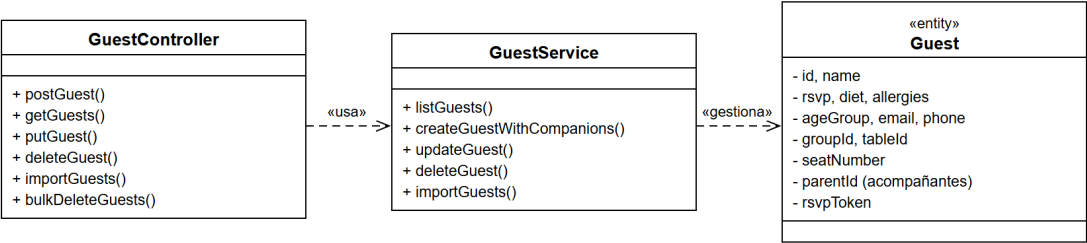


Figura 7.2. Diagrama de clases del módulo de invitados, cuya estructura es común a todos los módulos. *Fuente: elaboración propia.*

7.2. Modelo de datos

El modelo de datos es **relacional** y está **enraizado en el usuario**: un usuario posee una o varias bodas, y toda la información del sistema (invitados, grupos, mesas, eventos, tareas, presupuesto y proveedores) cuelga de una boda. Esta jerarquía es la que permite el **aislamiento entre bodas**: cada entidad de dominio referencia la boda a la que pertenece, lo que hace posible acotar cualquier consulta al propietario correspondiente (RNF-01).

Los **borrados se propagan en cascada** desde la boda hacia todas sus entidades hijas (y desde el usuario hacia sus bodas), de modo que eliminar una boda deja el sistema en un estado consistente sin registros huérfanos. La Figura 7.3 muestra las entidades principales del dominio y sus relaciones.

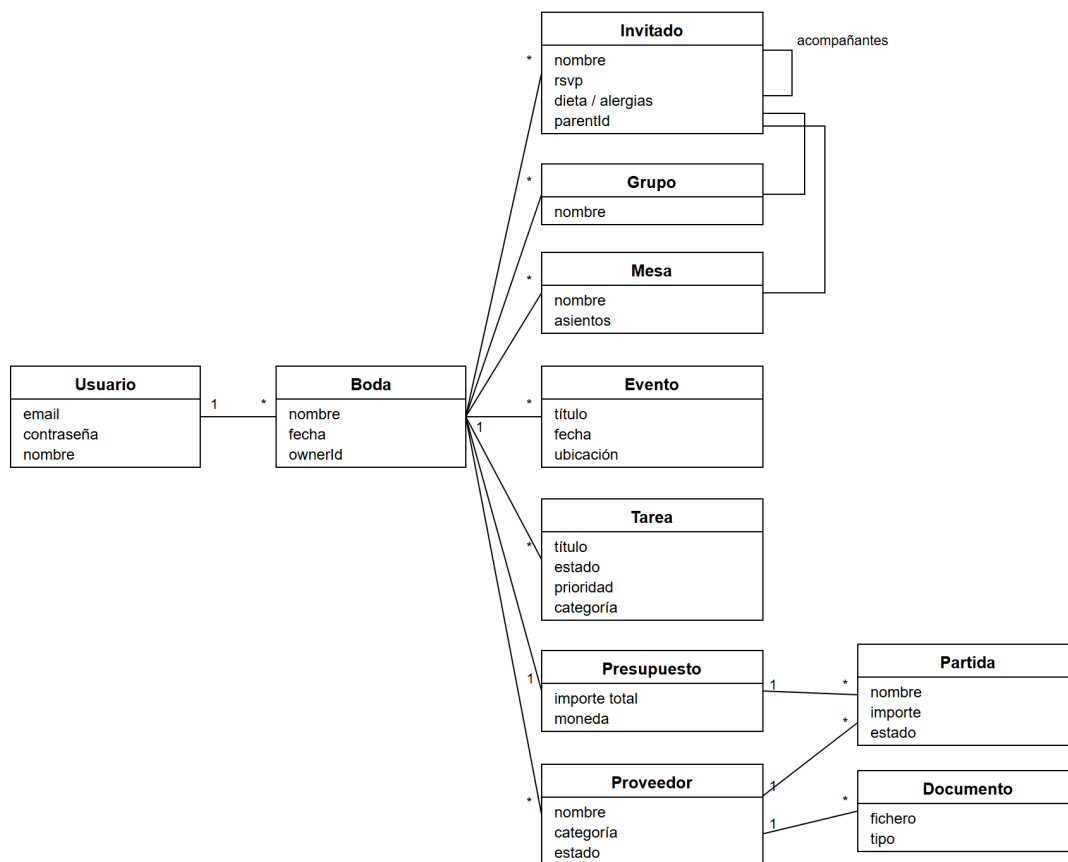


Figura 7.3. Modelo de datos del dominio. *Fuente: elaboración propia.*

Las principales entidades y decisiones de diseño son:

- **Usuario y Boda.** El usuario guarda sus credenciales (con la contraseña cifrada) y posee sus bodas. La boda almacena su nombre y fecha, y actúa como raíz del resto del modelo.
- **Invitado.** Es la entidad más rica. Además de sus datos (nombre, correo, teléfono, asistencia, dieta, alergias, grupo de edad, notas), incorpora una **relación reflexiva**: un invitado principal puede tener varios **acompañantes**, modelados como invitados que apuntan a su «padre». Referencia opcionalmente un grupo y una mesa con su número de asiento, y dispone de un **token único** para el enlace público de confirmación de asistencia.

- **Grupo y Mesa.** Agrupan invitados. La mesa define su número de asientos. Para garantizar que **un asiento no se asigne dos veces**, el modelo impone una **restricción de unicidad** sobre la pareja (mesa, número de asiento).
- **Evento y Tarea.** Modelan la agenda y el listado de tareas. La tarea utiliza **enumerados** para su prioridad, estado y categoría, lo que garantiza la integridad de esos valores a nivel de base de datos.
- **Presupuesto y partidas de gasto.** La boda tiene un presupuesto global (relación uno a uno) y un conjunto de partidas. Cada partida registra importes estimado, real y pagado, su estado y su categoría, y puede **vincularse opcionalmente a un proveedor** (con desvinculación automática si el proveedor se elimina), conservando además un campo de proveedor en texto libre como respaldo.
- **Proveedor y documentos.** El proveedor guarda categoría, estado de contratación, contacto y precios, y puede tener asociados varios **documentos** (contratos, presupuestos), de los que se almacenan sus metadatos.

De forma auxiliar, el modelo incluye dos tablas ligadas al usuario para la gestión segura de la sesión —una para los **tokens de refresco** activos y otra para los **tokens de recuperación de contraseña**— que se describen en §7.4 y no se representan en la Figura 7.3 por no pertenecer al dominio de la boda.

El uso de un **esquema declarativo** con generación automática del cliente aporta **tipado estático de extremo a extremo** en el backend y un **sistema de migraciones** versionado, lo que reduce los errores de acceso a datos y facilita la evolución del esquema.

7.3. Comunicación y API REST

La comunicación entre cliente y servidor se realiza mediante una **API REST** que intercambia **JSON** sobre HTTP. Todos los puntos de acceso cuelgan del prefijo `/api` y, salvo los públicos (salud, autenticación y confirmación de asistencia), exigen un *token* de sesión en la cabecera `Authorization: Bearer`.

Los recursos se organizan por dominio y utilizan los verbos HTTP de forma convencional (`GET` para consultar, `POST` para crear, `PUT/PATCH` para actualizar y `DELETE` para eliminar). El **alcance por boda** se transmite como parámetro (`?weddingId=...` o en la ruta), y el servidor acota cada operación a esa boda tras verificar su propiedad. La Tabla 7.1 recoge una muestra representativa de la API.

Tabla 7.1. Muestra de puntos de acceso de la API.

Verbo y ruta	Función
POST /api/auth/register · /login · /refresh · /logout	Alta, inicio de sesión, refresco y cierre de sesión
GET/POST/PUT/DELETE /api/guests	Gestión de invitados
POST /api/guests/import · /guests/bulk-delete	Importación y borrado masivo
GET/POST/DELETE /api/tables · PUT /api/tables/:id/seats/:n/assign	Mesas y asignación de asientos
GET/POST/PUT/DELETE /api/tasks · /events · /budget/items · /providers	Tareas, agenda, presupuesto y proveedores
GET /api/dashboard · /notifications	Panel de seguimiento y avisos
GET/POST /api/public/rsvp/:token	Confirmación pública de asistencia (sin sesión)
POST /api/ai/tasks/suggest · /ai/seating/suggest · /ai/budget/suggest	Asistencias con IA

Un aspecto de diseño relevante es el **tratamiento uniforme de errores** (RNF-22). El servidor dispone de un manejador central que convierte cualquier fallo en una respuesta JSON homogénea con la forma `{ error }`: los errores de validación se devuelven como **400** con un mensaje legible en español, los conflictos conocidos de la base de datos se mapean a sus códigos HTTP (por ejemplo, un valor duplicado a **409** y un registro inexistente a **404**), y cualquier error inesperado se enmascara como **500 genérico** sin filtrar detalles internos en producción. En el cliente, la capa de acceso a la API extrae ese campo `error` del cuerpo, de modo que los componentes muestran siempre un mensaje limpio al usuario.

7.4. Autenticación y autorización

Autenticación. La sesión se basa en **JSON Web Tokens (JWT)** (Jones et al., 2015) con dos piezas: un *token* de **acceso** de vida corta, que autoriza cada petición, y un *token* de **refresco** de vida larga, que permite renovar el acceso sin volver a introducir las credenciales. Cuando el acceso expira, el cliente solicita de forma transparente un nuevo par de *tokens* mediante el *token* de refresco (RF-04), lo que evita que la expiración del acceso expulse al usuario al inicio de sesión.

Para poder **invalidar sesiones de forma efectiva** (RF-05), cada *token* de refresco lleva un identificador único que se **persiste** en la base de datos. Al renovar, el servidor comprueba que ese identificador siga vigente y **rota** el par de *tokens*, eliminando el anterior; de este modo un *token* de refresco antiguo deja de ser válido en cuanto se usa uno nuevo. El cierre de sesión revoca el identificador presentado. La **recuperación de contraseña** (RF-06) se apoya en un *token* de un solo uso del que se guarda únicamente su **huella (hash)** con caducidad limitada; al restablecer la contraseña, el *token* se consume y se revocan todas las sesiones activas del usuario.

La Figura 7.4 resume el flujo de acceso y renovación de sesión.

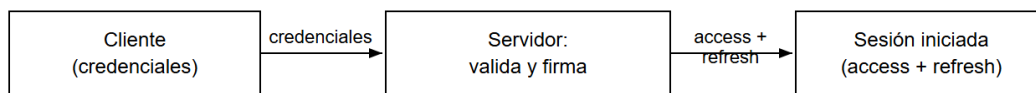
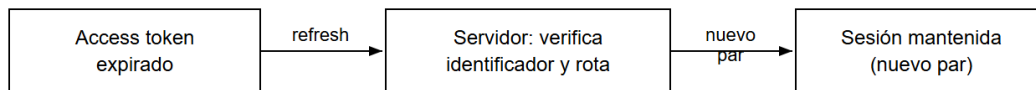
Inicio de sesión**Renovación de sesión**

Figura 7.4. Flujo de autenticación y renovación de la sesión. *Fuente: elaboración propia.*

Autorización. El sistema no distingue roles ni permisos entre usuarios: la autorización se basa por completo en la **propiedad de la boda** (RNF-01). Se aplica en dos frentes complementarios. Para las operaciones con alcance de boda (listar, crear, importar, acciones masivas), un **middleware** verifica que el `weddingId` de la petición pertenezca al usuario del *token*, rechazando con **404** cualquier acceso ajeno para no revelar la existencia del recurso. Para las operaciones sobre un recurso concreto (por identificador), las consultas de servicio se **acotan por propietario**, cerrando así la posibilidad de acceder a recursos de otra boda manipulando su identificador. Esta doble barrera fue una de las decisiones de diseño más importantes del proyecto, al ser la base del aislamiento entre bodas.

7.5. Estructura de navegación

La navegación del frontend se resuelve con un enrutador de cliente que distingue dos grandes zonas:

- **Rutas públicas**, accesibles sin sesión: inicio de sesión, registro, recuperación y restablecimiento de contraseña, y la **página pública de confirmación de asistencia** (`/rsvp/:token`). Estas rutas usan una disposición mínima sin la estructura de la aplicación.
- **Rutas protegidas**, envueltas en un componente de guarda que exige sesión válida y en la **disposición principal** (cabecera con selector de boda, cuenta atrás, avisos y perfil; y menú lateral de navegación). De aquí cuelgan las páginas de los módulos: inicio (panel), invitados, tareas, agenda, presupuesto, proveedores y ajustes.

El **menú de navegación** se define en un único lugar y actúa como **fuentes única de verdad**, de la que se derivan tanto el menú lateral de escritorio como el menú desplegable en móvil. De forma coherente, en el servidor las **rutas públicas se registran antes** que las protegidas, ya que estas últimas montan la exigencia de sesión en su raíz e interceptarían cualquier petición posterior; este orden fue necesario para que el enlace público de confirmación de asistencia conviviera con el resto de la API.

7.6. Diseño de la interfaz

La interfaz se ha construido sobre un **sistema de diseño** basado en un framework de utilidades CSS y una biblioteca de componentes accesibles, lo que garantiza coherencia visual y comportamiento uniforme en todos los módulos. Los colores, la tipografía y los espaciados se definen como **variables de diseño (tokens)**, lo que permite un **modo claro y un modo oscuro** consistentes en toda la aplicación.

Se ha adoptado una **identidad visual de estilo editorial** acorde con la temática de una boda: una paleta cálida, tipografía serif para los titulares y tarjetas de trazo fino. El objetivo fue que la aplicación resultara clara y cuidada —coherente con la ilusión de organizar una boda— sin sacrificar la claridad funcional.

El diseño se apoya en **componentes reutilizables** que unifican el comportamiento en toda la aplicación: diálogos, tarjetas de métricas, formularios y, en particular, un **diálogo de confirmación** común a todas las acciones destructivas y un sistema de **avisos (toasts)** que informa del resultado de cada operación (RNF-23). Esta reutilización reduce la duplicación y asegura que acciones equivalentes se comporten igual en cualquier pantalla.

La interfaz es **responsive** (RNF-20): en pantallas pequeñas, el menú lateral se oculta y su contenido se ofrece a través de un panel deslizante accesible desde la cabecera, y las tablas anchas incorporan desplazamiento horizontal. Toda la aplicación está **localizada en español**, con formatos de fecha y moneda `es-ES/EUR` (RNF-21). Las capturas de las distintas pantallas se presentan en el capítulo 8 (Implementación) y, de forma guiada, en el Anexo B (Manual de usuario).

8. IMPLEMENTACIÓN

Este capítulo describe cómo se ha llevado a la práctica el diseño del capítulo 7. No se detalla el código archivo por archivo —lo que resultaría extenso y poco útil— sino que se seleccionan los aspectos con mayor relevancia técnica: la estructura e inicialización del proyecto, la parte del cliente, la parte del servidor, un conjunto de funcionalidades destacadas por su lógica y, por último, la integración de la inteligencia artificial. Los fragmentos de código se incluyen únicamente cuando ayudan a explicar una decisión; las capturas de las pantallas se recogen en este capítulo y, de forma guiada, en el Anexo B.

8.1. Estructura e inicialización del proyecto

El proyecto se organiza en un único repositorio con dos carpetas independientes, `frontend` y `backend`, cada una con sus propias dependencias y scripts. Esta separación permite desarrollar, probar y desplegar ambas partes por separado.

Inicialización del backend. El servidor se construye sobre Express. Al arrancar, encadena una serie de *middlewares* en un orden significativo: cabeceras de seguridad, control de origen (CORS) según el entorno, registro de peticiones, análisis del cuerpo JSON, limitación de frecuencia en las rutas de autenticación y, finalmente, el enrutador principal montado bajo `/api`. El **manejador central de errores** se registra en último lugar, de modo que captura cualquier fallo producido en las capas anteriores. La configuración sensible (cadena de conexión, secretos de firma, credenciales de correo y de IA) se externaliza en variables de entorno que se **validan al arranque**: en producción, la ausencia o el uso de valores inseguros en los secretos de sesión aborta el inicio (RNF-02).

Acceso a datos con Prisma. La persistencia es uno de los puntos donde la elección tecnológica tuvo mayor impacto en la productividad, por lo que merece un detalle específico. Prisma sigue un enfoque *schema-first*: el modelo de datos se declara en un único fichero de esquema y, a partir de él, el comando `prisma generate` produce automáticamente un cliente de acceso a datos con **tipado estático de extremo a extremo**, mientras que `prisma migrate` deriva y aplica las **migraciones** versionadas sobre la base de datos. El siguiente fragmento del esquema muestra la entidad de invitados, con su relación reflexiva (un invitado principal y sus acompañantes), la propagación de borrados en cascada desde la boda y la restricción de unicidad que impide asignar dos veces el mismo asiento:

```

model Guest {
  id          String      @id @default(cuid())
  name        String
  weddingId   String
  wedding     Wedding     @relation(fields: [weddingId], references: [id], onDelete: Cascade)
  groupId     String?
  tableId     String?
  seatNumber  Int?
  role        GuestRole   @default(PRIMARY)
  rsvp        RsvpStatus  @default(PENDING)
  allergies   String[]    @default([])
  parentId    String?
  parent      Guest?      @relation("GuestCompanions", fields: [parentId], references: [id],
onDelete: Cascade)
  companions  Guest[]     @relation("GuestCompanions")

  @@unique([tableId, seatNumber])
}

```

Cuadro 8.1. Declaración del modelo de invitados (`Guest`) en el esquema de Prisma. *Fuente: elaboración propia.*

A partir de esta declaración, el cliente generado ofrece métodos cuyos nombres y tipos se derivan del propio modelo (`prisma.guest.findMany`, `create`, `update`...): referirse a un campo inexistente o asignar un tipo incorrecto se detecta en **tiempo de compilación**, no en ejecución. Los enumerados del dominio (`GuestRole`, `RsvpStatus`, etc.) se declaran en el mismo esquema, lo que garantiza su integridad a nivel de base de datos, y las claves foráneas fijan el comportamiento ante borrados (cascada hacia los hijos de la boda, `SetNull` cuando la relación es opcional). Frente a ORM basados en decoradores como `TypeORM`, este enfoque reduce el esquema, los tipos de `TypeScript` y la estructura real de la base de datos a una **única fuente de verdad**. Un script de **semilla** completa la inicialización creando datos de demostración (un usuario de prueba con su boda) para facilitar el desarrollo y las pruebas.

Inicialización del frontend. La aplicación se construye con `Vite` y `React`. El punto de entrada monta el árbol de componentes —proveedor de tema, enrutador y sistema de avisos— y delega en el enrutador la selección de la página según la URL. A partir de ahí, cada página consume los servicios de dominio para obtener y modificar los datos.

La estructura de carpetas del proyecto es la siguiente:

```

wedding-planner-ai/
├── backend/                # API REST (Express + TypeScript + Prisma)
│   ├── prisma/            # esquema, migraciones y semilla de datos
│   └── src/
│       ├── routes/        # definición de la API
│       ├── controllers/   # validación de la entrada
│       ├── services/      # lógica de negocio y acceso a datos
│       ├── middleware/    # autenticación, propiedad de boda y errores
│       └── utils/         # utilidades (contraseñas, tokens, correo)
├── frontend/              # aplicación de página única (React + TypeScript + Vite)
│   └── src/
│       ├── pages/         # páginas de la aplicación
│       ├── components/    # componentes reutilizables
│       ├── services/      # acceso a la API por dominio
│       └── lib/           # capa de API, autenticación y utilidades

```

Cuadro 8.2. Estructura de carpetas del proyecto. *Fuente: elaboración propia.*

8.2. Parte del cliente

Capa única de acceso a la API. Todas las llamadas de red pasan por un módulo que expone funciones equivalentes a los verbos HTTP (`apiGet`, `apiPost`, `apiPut`, `apiPatch`, `apiDelete`, además de utilidades para subir ficheros y descargar binarios). Esta capa centraliza tres responsabilidades transversales: **inyectar el *token*** de sesión en la cabecera de cada petición, **renovar la sesión** de forma transparente cuando el servidor responde que el *token* de acceso ha expirado (reintentando la petición una sola vez y compartiendo una única renovación entre llamadas concurrentes), y **extraer el mensaje de error** del cuerpo de la respuesta para que los componentes muestren siempre un texto limpio. Concentrar esta lógica en un único punto evita repetirla en cada pantalla y garantiza un comportamiento uniforme.

Sobre esa capa, **cada dominio dispone de su propio servicio** (invitados, tareas, presupuesto, etc.), que encapsula sus operaciones y coloca junto a ellas los tipos y las **etiquetas en español** de los valores enumerados (por ejemplo, la traducción de los estados de un proveedor). El **estado de sesión** —tokens y boda activa— se guarda en el almacenamiento local del navegador; antes de cualquier operación con alcance de boda, la aplicación recupera de ahí la boda seleccionada.

Navegación e interfaz. El enrutador distingue las rutas públicas de las protegidas (estas últimas envueltas en una guarda de sesión y en la disposición principal, con cabecera y menú lateral). El menú se declara en un único lugar y alimenta tanto la barra lateral de escritorio como el panel deslizante en móvil.

La reutilización de componentes es un pilar de la coherencia visual. Dos ejemplos transversales: un sistema de **avisos (toasts)** que confirma el resultado de cada acción, y un **diálogo de confirmación** común a todas las operaciones destructivas. El patrón de uso es homogéneo en toda la aplicación:

```
try {
  await apiDelete(`/tasks/${task.id}`);
  toast.success("Tarea eliminada");
} catch (e) {
  toastError(e); // muestra el mensaje limpio extraído por la capa de API
}
```

Cuadro 8.3. Patrón de borrado con aviso y manejo de errores en el cliente. *Fuente: elaboración propia.*

De este modo, acciones equivalentes se comportan igual en cualquier pantalla y el usuario recibe siempre una respuesta clara de lo que ha ocurrido (RNF-23).

8.3. Parte del servidor

La lógica del servidor sigue el flujo en capas descrito en el diseño: **ruta** → **controlador** → **servicio**. El controlador valida la entrada con un **esquema de Zod** y delega en el servicio, que contiene la lógica y el acceso a datos. El uso de Zod es representativo de una de las decisiones técnicas del proyecto: un mismo esquema valida los datos en tiempo de ejecución y, a la vez, **infiere el tipo de TypeScript** de la entrada ya validada, evitando declararlo por separado. Además, cuando la validación falla, Zod produce un

error que el manejador central traduce a una respuesta 400 con un mensaje legible (RNF-22):

```
const createGuestSchema = z.object({
  name: z.string().min(1),
  groupId: z.string().optional(),
  rsvp: z.enum(["PENDING", "CONFIRMED", "DECLINED"]).default("PENDING"),
  allergies: z.array(z.string()).default([]),
  companions: z.array(z.object({ name: z.string().min(1) })).default([]),
});

// En el controlador: valida y, si es correcto, delega en el servicio.
const data = createGuestSchema.parse(req.body); // data queda tipado
const guest = await guestService.createWithCompanions(weddingId, data);
```

Cuadro 8.4. Validación de la entrada con Zod en el controlador. *Fuente: elaboración propia.*

El siguiente fragmento ilustra el patrón en la capa de servicio con el alta de un invitado y sus acompañantes, que debe ser **atómica** (o se crean todos, o no se crea ninguno) y por tanto se resuelve en una **transacción**:

```
return prisma.$transaction(async (tx) => {
  const primary = await tx.guest.create({
    data: { weddingId, name, role: "PRIMARY", groupId, allergies },
  });
  if (companions.length) {
    await tx.guest.createMany({
      data: companions.map((c) => ({
        weddingId, parentId: primary.id, role: "COMPANION", name: c.name,
      })),
    });
  }
  return tx.guest.findUnique({
    where: { id: primary.id },
    include: { companions: true },
  });
});
```

Cuadro 8.5. Alta transaccional de un invitado y sus acompañantes. *Fuente: elaboración propia.*

Dos *middlewares* completan la capa de servidor. El de **propiedad de boda** (RNF-01) intercepta las operaciones con alcance de boda y rechaza con 404 cualquier `weddingId` que no pertenezca al usuario del *token*. El **manejador de errores** (RNF-22) traduce cualquier excepción a una respuesta JSON homogénea: errores de validación a 400 con mensaje en español, conflictos de la base de datos a su código HTTP correspondiente, y errores inesperados a un 500 genérico que no filtra detalles internos.

Por último, el servidor integra dos servicios externos. El **envío de correo** utiliza un transportador que se inicializa de forma perezosa: si hay credenciales SMTP configuradas las usa, y en caso contrario recurre a una **cuenta de prueba** que no entrega el correo a destinatarios reales pero devuelve un enlace de previsualización, lo que permite desarrollar y probar el envío de invitaciones sin infraestructura de correo. La **subida de documentos** de proveedores se gestiona con almacenamiento en disco, asignando a cada fichero un nombre único y validando su tipo y tamaño.

8.4. Funcionalidades destacadas

8.4.1. Invitados, acompañantes e importación

El módulo de invitados es el más rico del sistema. Además del alta transaccional de acompañantes ya mostrada, incorpora una **importación masiva** desde CSV que crea el grupo indicado si no existe, y un **borrado masivo** de invitados seleccionados.

La importación planteó un problema práctico relevante: al abrir un CSV en la versión española de Excel, los **acentos y la «ñ» se corrompían y todo el contenido caía en una sola columna**. La causa es doble: Excel espera la codificación local salvo que el fichero declare UTF-8 mediante una marca de orden de bytes (BOM), y en la configuración regional española el separador de columnas por defecto es el punto y coma, no la coma. La solución fue anteponer la marca UTF-8 y emplear el punto y coma como separador, de modo que el mismo fichero se ve correctamente —columnas y acentos— sin que el usuario tenga que configurar nada. El analizador de importación, además, ignora la fila de cabecera y las líneas auxiliares para que una plantilla descargada pueda rellenarse y reimportarse sin cambios.

La Figura 8.1 muestra la pantalla de gestión de invitados.

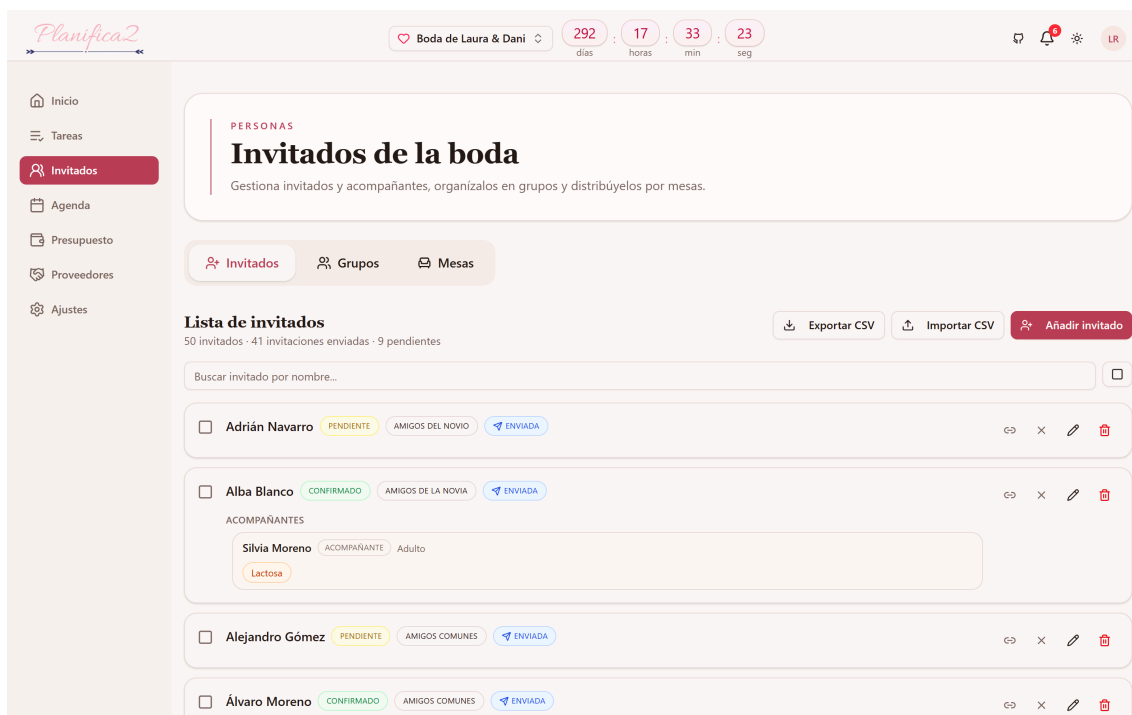


Figura 8.1. Pantalla de gestión de invitados, con acompañantes, estado de asistencia e invitación. *Fuente: elaboración propia.*

8.4.2. Grupos y mesas: la distribución visual

La asignación de invitados a mesas fue uno de los puntos que más iteración requirió, no por la lógica de datos sino por **cómo representarla visualmente** de forma que resultara clara. Se optó por un **mapa de mesas** en el que cada mesa muestra sus asientos y el usuario asigna un invitado seleccionado a un asiento concreto. A nivel de datos, la integridad de la asignación se garantiza con la restricción de unicidad sobre la pareja (mesa, asiento) descrita en el diseño, de modo que es imposible colocar a dos personas

en el mismo sitio. Se añadieron además acciones para **liberar un asiento** o **vaciar una mesa** por completo.

La Figura 8.2 muestra el mapa de mesas con la distribución de invitados.

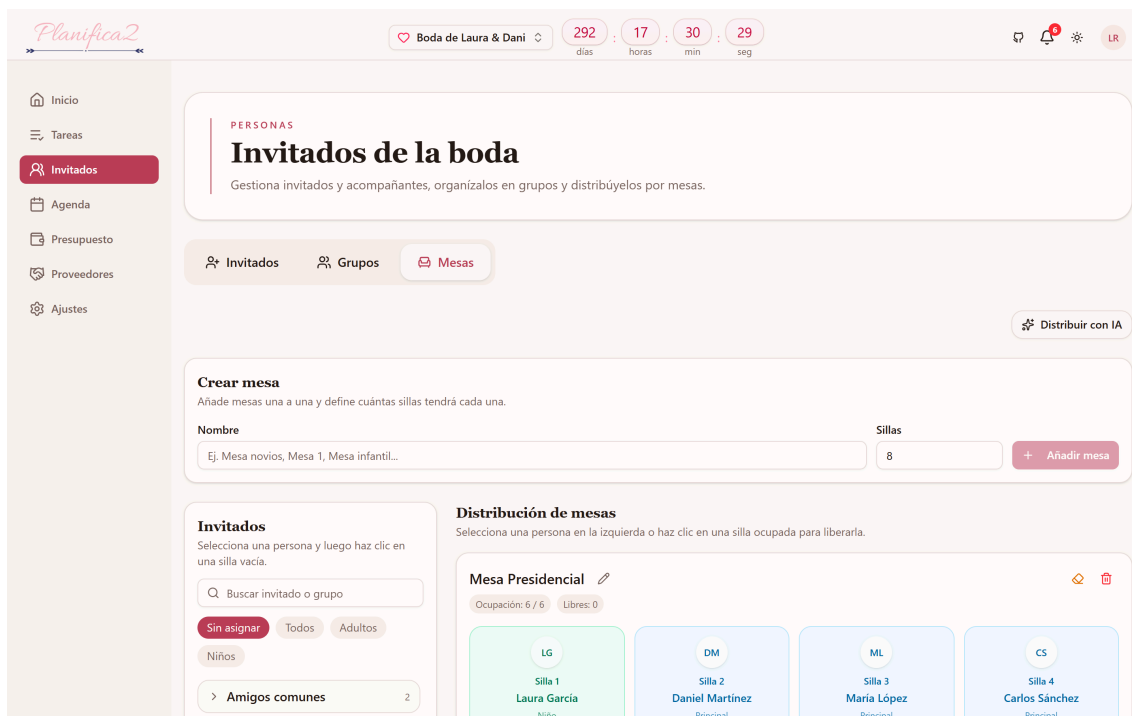


Figura 8.2. Mapa de mesas con la asignación de invitados a asientos. *Fuente: elaboración propia.*

8.4.3. Presupuesto y proveedores

El módulo de presupuesto permite definir un importe global y gestionar partidas de gasto con su categoría, importes estimado, real y pagado, y estado. La información se resume en **gráficas** de reparto por categoría y de seguimiento del gasto. Cada partida puede **vincularse a un proveedor**, lo que conecta ambos módulos y evita duplicar información. Los proveedores, a su vez, admiten **documentos adjuntos** (contratos o presupuestos) que se suben, listan, descargan y eliminan desde la ficha del proveedor.

La Figura 8.3 muestra la gestión del presupuesto.

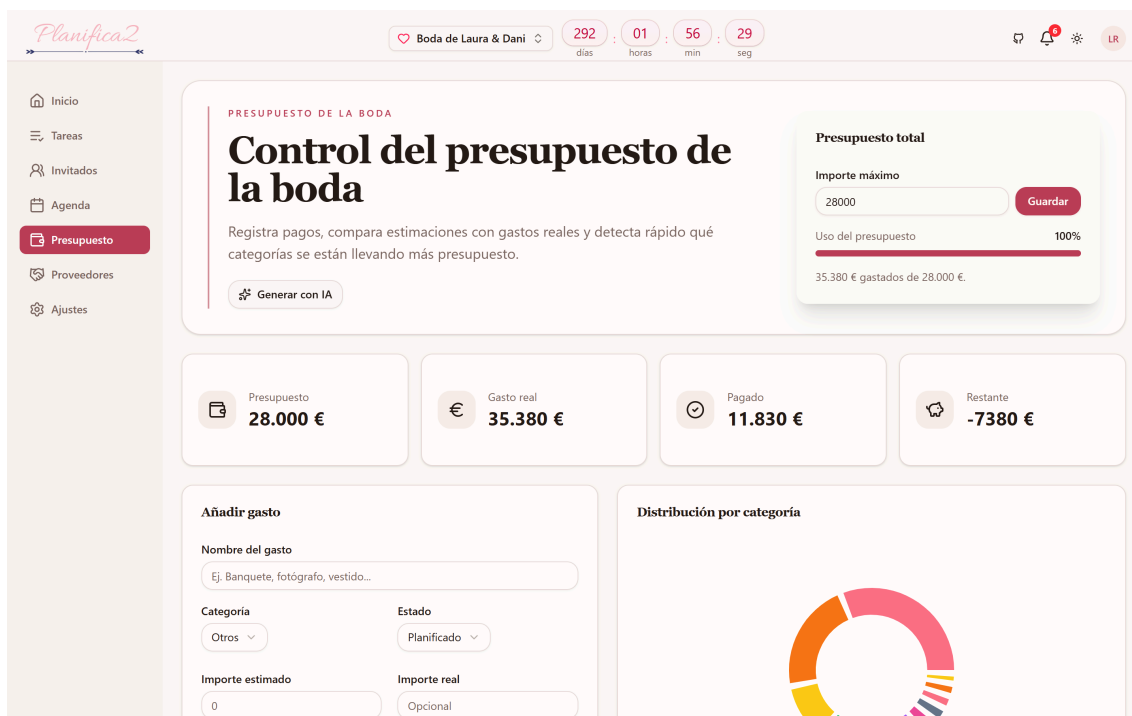


Figura 8.3. Gestión del presupuesto con sus gráficas de reparto y seguimiento. *Fuente: elaboración propia.*

8.4.4. Panel de seguimiento y avisos

La página de inicio ofrece un **panel** que agrega el estado de la boda: distribución de invitados por asistencia y edad, ocupación de mesas, progreso de tareas, estado del presupuesto, invitaciones enviadas y próximos eventos. De forma complementaria, un sistema de **avisos** calcula en el momento las tareas sin completar y los pagos pendientes cuya fecha está vencida o próxima, y los presenta en una campana de notificaciones en la cabecera. Ambos se calculan siempre acotados a la boda del usuario (RNF-01).

La Figura 8.4 muestra el panel de seguimiento de la boda.

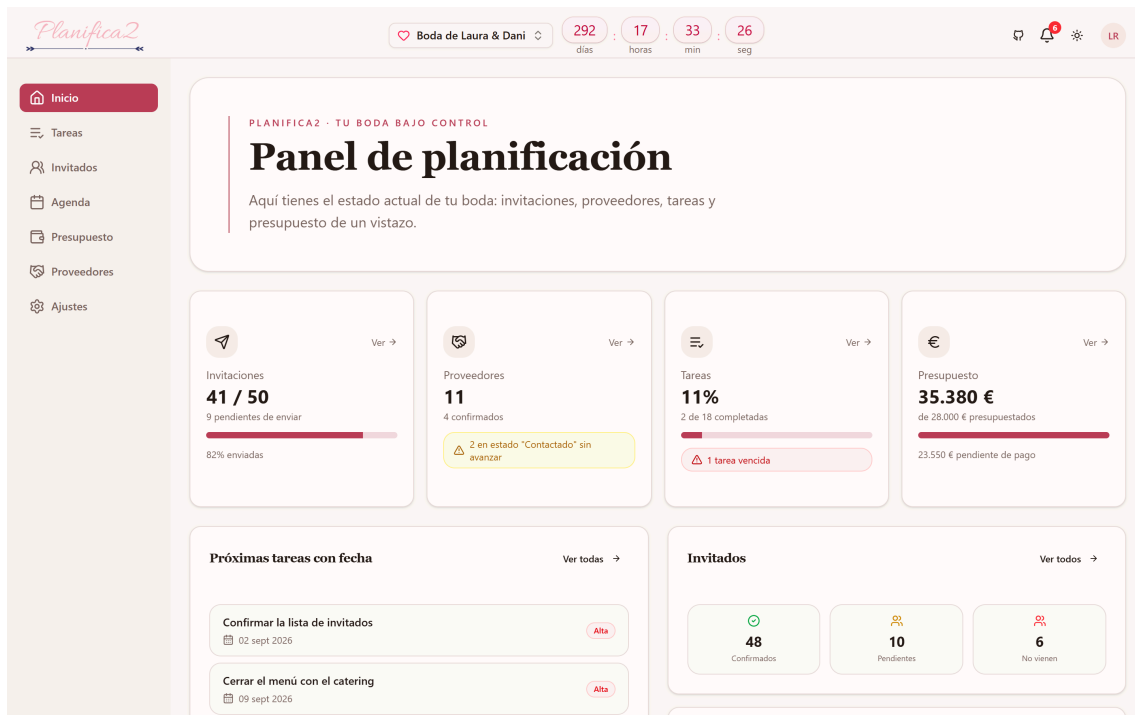


Figura 8.4. Panel de seguimiento de la boda. Fuente: elaboración propia.

8.5. Integración de la inteligencia artificial

Las tres asistencias con IA comparten una misma infraestructura, diseñada en torno a dos principios: **fiabilidad del formato** y **control del usuario**.

La fiabilidad se consigue con una capa común que solicita a la IA una respuesta en **formato estructurado** (un esquema JSON estricto) y **valida** el resultado contra ese mismo esquema antes de devolverlo. Si la respuesta no cumple el formato esperado, no se utiliza. Cada asistente aporta su esquema y construye el contexto de la boda a partir de la base de datos: por ejemplo, el asistente de tareas recibe la fecha de la boda, el número de invitados y las tareas ya existentes para no repetirlas.

Esta fiabilidad se apoya en una técnica reciente de la API de OpenAI, los **structured outputs**: la llamada se realiza con `response_format` de tipo `json_schema` en modo `strict`, lo que **obliga al modelo a devolver un JSON que se ajusta exactamente al esquema indicado**, en lugar de texto libre que habría que interpretar. Aun así, como capa de defensa adicional, la respuesta se vuelve a validar en el servidor con el esquema de Zod correspondiente antes de usarse (IA-91); si no supera esa validación, se descarta y se informa al usuario sin exponer detalles internos (IA-92). Toda esta lógica se concentra en una única función `generateStructured`, que cada asistente reutiliza aportando su esquema y su contexto:

```
const completion = await openai.chat.completions.create({
  model: env.ai.model,
  temperature: 0.4,
  messages: [
    { role: "system", content: opts.system },
    { role: "user", content: opts.user },
  ],
  response_format: {
    type: "json_schema",
    json_schema: { name: opts.schemaName, schema: opts.jsonSchema, strict: true },
  },
});

// Segunda barrera: validar la respuesta con Zod antes de devolverla.
const validated = opts.schema.safeParse(JSON.parse(content));
if (!validated.success) {
  throw { status: 502, message: "La respuesta de la IA no cumple el formato esperado." };
}
return validated.data;
```

Cuadro 8.6. Llamada a la IA con salida estructurada (`json_schema`) y validación con Zod. *Fuente: elaboración propia.*

El control del usuario se garantiza porque las funciones de IA **nunca escriben en la base de datos directamente**: devuelven **propuestas editables** que el usuario revisa — pudiendo descartar los elementos que no desee— y solo entonces se persisten mediante los mismos endpoints de alta que el resto de la aplicación. Así, la generación de tareas confirmadas reutiliza el alta de tareas, y la de presupuesto y proveedores, sus respectivas altas.

La asistencia de **distribución de mesas** merece una mención especial. Como los identificadores de mesas e invitados son dinámicos y no pueden enumerarse por adelantado en el esquema, la propuesta de la IA podría contener referencias inválidas. Por ello, el servidor **valida y depura** la propuesta antes de mostrarla —descartando identificadores inexistentes, asientos fuera de capacidad y cualquier asiento o invitado repetido— y, cuando el usuario la acepta, la aplica en una **transacción** que respeta la restricción de unicidad de asientos.

Respecto a la seguridad y la robustez, la clave de acceso a la IA reside **exclusivamente en el backend** (IA-90); el cliente solo consulta un indicador de estado para mostrar u ocultar las funciones de IA. Ante un fallo, el sistema **degrada con elegancia**: informa de forma clara si la funcionalidad no está configurada y muestra un mensaje amable si la API falla, sin exponer detalles internos.

La Figura 8.5 muestra el diálogo de revisión de una propuesta generada por la inteligencia artificial.

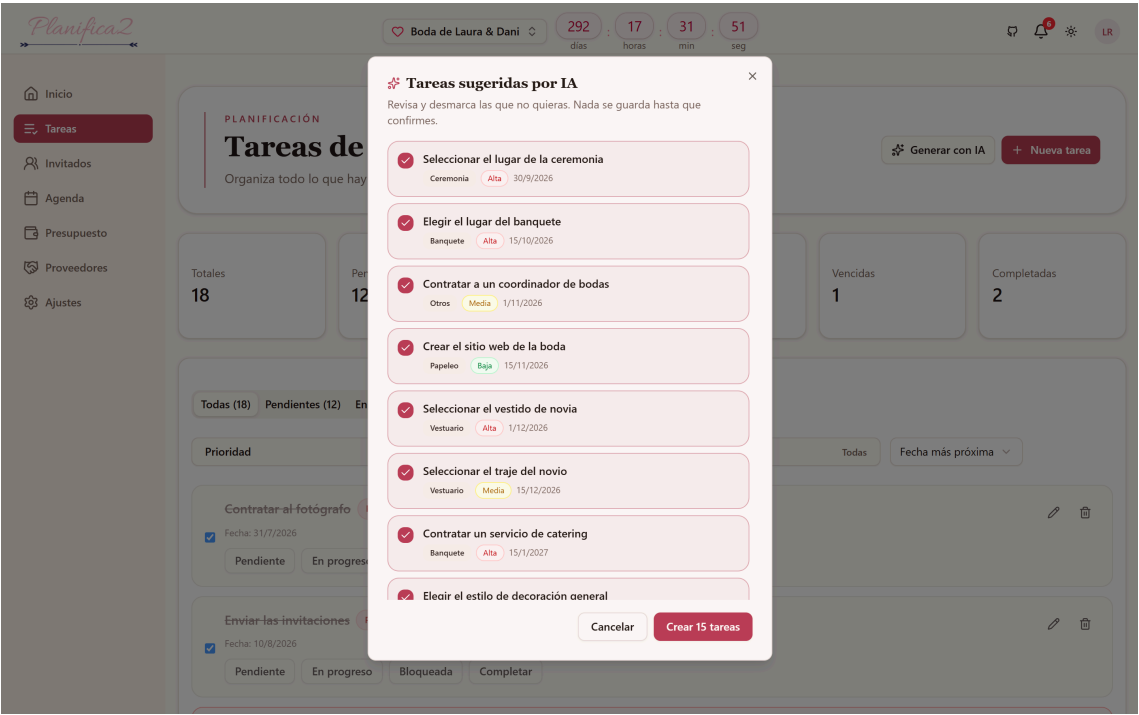


Figura 8.5. Diálogo de revisión de una propuesta de tareas generada por la inteligencia artificial, antes de confirmarla. *Fuente: elaboración propia.*

9. PRUEBAS Y VALIDACIÓN

Para garantizar el correcto funcionamiento del sistema se ha seguido una **estrategia de pruebas en varios niveles**: pruebas automatizadas —unitarias y de integración— sobre la lógica del servidor, pruebas funcionales manuales sobre la aplicación completa y un proceso de integración continua que ejecuta las validaciones de forma automática ante cada cambio. Este capítulo describe cada nivel y presenta los resultados obtenidos.

9.1. Estrategia de pruebas

Las pruebas se han organizado siguiendo el criterio habitual de la pirámide de pruebas: una base amplia de **pruebas unitarias** rápidas sobre las utilidades puras, un conjunto de **pruebas de integración** que ejercitan la API de extremo a extremo contra una base de datos real, y una capa de **verificación funcional manual** sobre la interfaz. Todo ello se apoya en un sistema de **integración continua** que actúa como red de seguridad frente a regresiones. Las pruebas automatizadas del backend se han implementado con el framework **Vitest** (Vitest, s.f.) —una alternativa moderna a Jest, integrada de forma nativa con Vite y TypeScript—, acompañado de la biblioteca **supertest** para las pruebas de la API.

9.2. Pruebas unitarias

Las pruebas unitarias verifican, de forma aislada y sin acceso a la base de datos, el comportamiento de las utilidades con lógica propia:

- **Cifrado de contraseñas**: que una contraseña se cifra y que su verificación posterior distingue correctamente entre la contraseña válida y una incorrecta.
- **Tokens de sesión (JWT)**: que un *token* firmado se verifica correctamente y que su contenido se recupera de forma fiable.
- **Plantilla de invitación por correo**: que el mensaje se genera con los datos esperados, comprobando además el **escape de HTML** para prevenir la inyección de contenido.
- **Manejador central de errores**: que un error de validación se traduce en una respuesta 400 con mensaje legible, que los errores con formato `{ status, message }` se respetan y que cualquier error inesperado se enmascara como 500 genérico (RNF-22).

9.3. Pruebas de integración

Las pruebas de integración ejercitan la API de extremo a extremo mediante peticiones HTTP reales contra la base de datos de desarrollo, limpiando después los datos que crean. El caso principal recorre el flujo **registro** → **boda** → **invitado** y verifica, entre otros, los siguientes comportamientos:

- El registro de un usuario crea automáticamente su primera boda.
- Un registro con un correo ya existente devuelve un conflicto (409).
- El alta y el listado de un invitado funcionan correctamente.

- Una petición sin *token* de sesión se rechaza con 401.
- Un intento de acceder a una boda ajena devuelve 404, validando el **aislamiento entre bodas** (RNF-01).
- Una petición con datos inválidos devuelve un 400 con un mensaje limpio en español (RNF-22).

9.4. Resultados de las pruebas automatizadas

La ejecución completa de la batería de pruebas del backend finaliza **sin errores**. La Tabla 9.1 resume los resultados, junto con la verificación de estilo (lint) y de compilación (build) de ambas partes del proyecto.

Tabla 9.1. Resultados de las validaciones automatizadas.

Validación	Resultado
Pruebas del backend (Vitest)	18 pruebas superadas en 5 ficheros
Análisis de estilo del backend (ESLint)	Sin errores
Compilación del backend (TypeScript)	Sin errores
Análisis de estilo del frontend (ESLint)	0 errores (36 avisos de estilo)
Compilación del frontend (build de producción)	Sin errores

Como limitación reconocida, el **frontend no dispone todavía de pruebas automatizadas**; su validación se apoya en la compilación con verificación de tipos y en las pruebas funcionales manuales descritas a continuación. Esta carencia se recoge como línea de trabajo futura (capítulo 11).

9.5. Pruebas funcionales manuales

De forma complementaria a las pruebas automatizadas, cada funcionalidad se ha verificado manualmente sobre la interfaz a medida que se implementaba y en una revisión final sobre la aplicación completa. Para ello se utilizó un conjunto de datos de demostración representativo (una boda con 64 invitados, grupos, mesas, tareas, agenda, presupuesto y proveedores). La Tabla 9.2 resume las comprobaciones funcionales por módulo.

Tabla 9.2. Verificación funcional por módulo.

Módulo	Comprobaciones
Autenticación	Registro, inicio y cierre de sesión, renovación transparente y recuperación de contraseña
Invitados	Alta con acompañantes, edición, borrado individual y masivo, importación y exportación CSV, envío de invitaciones
Grupos y mesas	Alta de grupos y mesas, asignación de invitados a asientos, liberación y vaciado
Tareas y agenda	Alta, edición y borrado; filtros; vista de calendario y cuenta atrás
Presupuesto y proveedores	Partidas, gráficas, vínculo proveedor-partida y documentos adjuntos
Panel y avisos	Agregados del panel y notificaciones de tareas y pagos próximos o vencidos
Asistencias con IA	Generación de propuestas de tareas, distribución de mesas y presupuesto, con revisión y confirmación por el usuario

Las capturas que documentan estas comprobaciones se recogen en el capítulo 8 y, de forma guiada, en el Anexo B.

9.6. Integración continua

El proyecto incorpora un flujo de **integración continua** (GitHub Actions) que se ejecuta ante cada cambio en la rama principal y que actúa como red de seguridad frente a regresiones. Se compone de dos trabajos: uno para el **backend** —que levanta una base de datos PostgreSQL de servicio, instala dependencias, genera el cliente de datos, ejecuta el análisis de estilo, compila, aplica las migraciones y lanza las pruebas— y otro para el **frontend** —que instala dependencias, ejecuta el análisis de estilo y compila—. Mantener ambos trabajos en verde es condición para integrar cualquier cambio.

9.7. Trazabilidad entre requisitos y pruebas

La Tabla 9.3 relaciona una selección de requisitos con la prueba que verifica su cumplimiento, cerrando el ciclo entre lo especificado (capítulo 6) y lo comprobado.

Tabla 9.3. Trazabilidad entre requisitos y pruebas.

Requisito	Prueba que lo verifica
RF-01, RF-02, RF-03	Integración: registro, boda automática, acceso sin token (401)
RF-10, RF-11	Integración: alta y listado de invitados
RF-80	Unitaria: generación de la plantilla de invitación
RNF-01	Integración: acceso a boda ajena (404)
RNF-02	Unitarias: cifrado de contraseñas y tokens
RNF-22	Unitaria e integración: errores 400/409/500 con mensaje limpio
IA-01, IA-02, IA-03	Funcional: generación, revisión y aplicación de propuestas
Resto de RF	Funcional: verificación por módulo (Tabla 9.2)

10. RESULTADOS

Este capítulo evalúa en qué medida se han alcanzado los objetivos planteados en el capítulo 3. Para ello se recuperan los objetivos específicos, se relacionan con los requisitos que los materializan (capítulo 6) y con las pruebas que los verifican (capítulo 9), y se valora su grado de cumplimiento. Se describen además los resultados obtenidos, los resultados no previstos inicialmente y las limitaciones detectadas.

10.1. Grado de cumplimiento de los objetivos

La Tabla 10.1 resume el estado de cada objetivo específico al finalizar el proyecto, apoyándose en los requisitos asociados y en su verificación.

Tabla 10.1. Grado de cumplimiento de los objetivos específicos.

Objetivo	Requisitos	Estado	Evidencia
OE-1. Cuenta y bodas	RF-01...RF-09, RNF-01/02	Alcanzado	Registro, inicio de sesión y gestión de varias bodas aisladas; verificado por pruebas
OE-2. Invitados	RF-10...RF-16, RF-80/81	Alcanzado	Alta con acompañantes, RSVP, invitaciones y confirmación pública (Fig. 8.1)
OE-3. Grupos y mesas	RF-20...RF-24	Alcanzado	Grupos y mapa de mesas con asignación de asientos (Fig. 8.2)
OE-4. Tareas y agenda	RF-30, RF-31, RF-40, RF-41	Alcanzado	Gestión de tareas y calendario de eventos
OE-5. Gestión económica	RF-50...RF-52, RF-60...RF-62	Alcanzado	Presupuesto con gráficas, proveedores y documentos (Fig. 8.3)
OE-6. Seguimiento global	RF-70, RF-71	Alcanzado	Panel de seguimiento y avisos (Fig. 8.4)
OE-7. Asistencias con IA	IA-01...IA-03, IA-90...IA-92	Alcanzado	Generación real de propuestas de tareas, mesas y presupuesto (Fig. 8.5)
OE-8. Calidad	RNF-03, RNF-10...RNF-13, RNF-20...RNF-23	Alcanzado (con matices)	18 pruebas del backend en verde, integración continua, interfaz responsive y localizada; sin pruebas automatizadas de frontend

Los **ocho objetivos específicos se han alcanzado**, y con ellos el objetivo general de disponer de una aplicación web integral para la organización de una boda con asistencias de inteligencia artificial. El único matiz relevante afecta a OE-8: la calidad se ha atendido en seguridad, usabilidad, robustez, pruebas del backend e integración continua, pero el frontend no cuenta todavía con pruebas automatizadas propias.

10.2. Resultados de la aplicación

El resultado principal es una **aplicación funcional y completa** que cubre el ciclo de organización de una boda. Para comprobar su comportamiento con un volumen realista

se cargó un conjunto de datos de demostración correspondiente a una boda con **64 invitados** (50 principales y 14 acompañantes), 6 grupos, 9 mesas, 18 tareas, 7 eventos, 11 proveedores y un presupuesto con 12 partidas.

Sobre ese conjunto, el **panel de seguimiento** (Fig. 8.4) agrega correctamente el estado de la boda —invitaciones enviadas, proveedores, progreso de tareas, estado del presupuesto, reparto de invitados por asistencia y próximas tareas—, y la **cuenta atrás** refleja el tiempo restante hasta la fecha de la boda. Los distintos módulos operan de forma coherente sobre esos datos, como muestran las capturas del capítulo 8 y del Anexo B.

Especial mención merecen las **asistencias de inteligencia artificial**, que constituían el elemento diferenciador del proyecto. La Figura 8.5 muestra una **generación real** del asistente de tareas: a partir del contexto de la boda, la IA propuso un conjunto de tareas con su categoría, prioridad y fecha, presentadas como propuestas editables que el usuario puede revisar y confirmar antes de crearlas. Las tres asistencias —tareas, distribución de mesas y presupuesto— funcionan siguiendo este mismo principio de propuesta editable bajo control del usuario.

Desde el punto de vista de la **calidad técnica**, la batería de pruebas del backend se ejecuta sin errores (18 pruebas superadas), y tanto el análisis de estilo como la compilación de ambas partes del proyecto finalizan correctamente (capítulo 9).

10.3. Resultados no previstos

Durante el desarrollo surgieron mejoras que, sin estar planteadas explícitamente al inicio, aportaron valor al producto final:

- Un **sistema de avisos (toasts)** y un **diálogo de confirmación** comunes a toda la aplicación, que unifican la retroalimentación al usuario y protegen las acciones destructivas.
- El **borrado masivo** de invitados y la **asignación masiva** a un grupo, que agilizan la gestión de listas grandes.
- Una **importación CSV enriquecida** que crea los grupos que no existan y resuelve los problemas de codificación al abrir los ficheros en Excel.

10.4. Desviaciones y limitaciones

El proyecto presenta también algunas limitaciones que conviene reconocer:

- **Ausencia de pruebas automatizadas en el frontend**, cuya validación se apoya en la compilación con verificación de tipos y en las pruebas funcionales manuales.
- **Dependencia de un servicio externo** para las asistencias de IA, con el coste asociado a su uso; el sistema está diseñado para degradar con elegancia si la IA no está disponible.
- El **envío de correo** en el entorno de desarrollo utiliza una cuenta de prueba; un despliegue real requiere configurar un servidor SMTP.

Estas limitaciones no comprometen los objetivos alcanzados y se retoman, como posibles vías de mejora, en el capítulo de conclusiones.

11. CONCLUSIONES Y LÍNEAS FUTURAS

En este capítulo se recogen las conclusiones del trabajo, contrastando lo conseguido con los objetivos planteados, y se exponen las posibles líneas de evolución futura del proyecto.

11.1. Conclusiones

El problema del que parte este trabajo es la dificultad de organizar una boda de forma centralizada. La organización de una boda exige coordinar durante meses numerosos aspectos interdependientes —invitados, mesas, tareas, presupuesto y proveedores—. Las herramientas existentes, orientadas a un modelo de negocio basado en el directorio de proveedores, no automatizan las decisiones organizativas internas de la pareja. El objetivo del proyecto era desarrollar una aplicación web que resolviera esta necesidad e incorporara, además, asistencias basadas en inteligencia artificial.

Para dar respuesta a este problema se ha desarrollado una aplicación web completa. Esta aplicación reúne en un único lugar la gestión de la cuenta y de varias bodas, los invitados y sus acompañantes con confirmación de asistencia, los grupos y la distribución en mesas, las tareas y la agenda, el presupuesto y los proveedores, y un panel de seguimiento con avisos. Sobre esta base se han integrado tres asistencias de inteligencia artificial —generación de tareas, distribución de mesas y reparto de presupuesto—, que operan siempre como propuestas editables bajo la revisión y confirmación del usuario. El sistema se ha construido sobre una arquitectura en capas, con autenticación basada en tokens y un modelo de autorización por propiedad de boda que garantiza el aislamiento entre bodas.

De forma resumida, las **funciones conseguidas** son las siguientes:

- Registro, inicio de sesión y gestión de la cuenta, con recuperación y cambio de contraseña.
- Gestión de varias bodas por usuario, con selector de boda activa y aislamiento entre ellas.
- Gestión de invitados y acompañantes, con estado de asistencia (RSVP), dieta y alergias, importación y exportación en CSV, y envío de invitaciones por correo electrónico.
- Confirmación pública de asistencia por parte del invitado mediante un enlace único, sin necesidad de cuenta.
- Organización de invitados en grupos y distribución visual en las mesas del banquete.
- Gestión de tareas (con prioridad, estado y categoría) y de una agenda de eventos con vista de calendario y cuenta atrás.
- Control del presupuesto con partidas de gasto y gráficas, y gestión de proveedores con documentos adjuntos.
- Panel de seguimiento que agrega el estado de la boda y avisos de tareas y pagos próximos o vencidos.

- Tres asistencias de inteligencia artificial —generación de tareas, distribución de mesas y reparto de presupuesto— como propuestas editables bajo control del usuario.

Las **tecnologías empleadas** se resumen en:

- **Cliente:** React, TypeScript, Vite, Tailwind CSS y shadcn/ui.
- **Servidor:** Node.js, Express, validación con Zod y ORM Prisma.
- **Base de datos:** PostgreSQL, ejecutada en un contenedor Docker.
- **Autenticación:** JSON Web Tokens (JWT) con renovación de sesión.
- **Inteligencia artificial:** API de OpenAI con salida estructurada (*structured outputs*).
- **Calidad y despliegue:** pruebas con Vitest y supertest, análisis de estilo con ESLint e integración continua con GitHub Actions.

En cuanto a los resultados, se han alcanzado los ocho objetivos específicos planteados y, con ellos, el objetivo general. La aplicación se ha validado con un conjunto de datos representativo de 64 invitados, y las asistencias de inteligencia artificial se han comprobado mediante generaciones reales. La calidad del sistema se respalda con 18 pruebas automatizadas del backend superadas sin errores, con una compilación y un análisis de estilo sin errores en ambas partes del proyecto, y con un flujo de integración continua. La principal limitación reconocida es la ausencia de pruebas automatizadas en el frontend.

Durante el desarrollo se afrontaron dificultades cuya resolución constituyó buena parte del aprendizaje del proyecto, entre ellas la representación visual de las mesas, el aislamiento entre bodas, la gestión de la expiración de la sesión y la codificación de los ficheros CSV. El trabajo ha permitido, asimismo, consolidar conocimientos en el desarrollo de aplicaciones web completas, desde el diseño de la arquitectura y el modelo de datos hasta la integración de servicios externos y de inteligencia artificial.

11.2. Líneas futuras

A partir del estado actual del proyecto y de sus limitaciones, se identifican las siguientes líneas de trabajo futuro:

- **Pruebas automatizadas del frontend.** Incorporar pruebas de componentes y de extremo a extremo que complementen las del backend y refuercen la prevención de regresiones.
- **Ampliación de las asistencias de IA.** Extender la inteligencia artificial a nuevas tareas (por ejemplo, sugerencias de proveedores o de agenda) y permitir refinar las propuestas de forma conversacional.
- **Aplicación móvil.** Ofrecer una versión para dispositivos móviles que facilite la consulta y la recepción de avisos en cualquier momento.
- **Exportación de documentos.** Permitir exportar a PDF elementos como la distribución de mesas, el calendario de eventos o el resumen del presupuesto.

- **Personalización del idioma.** Añadir soporte multi-idioma para llegar a un público más amplio.
- **Colaboración entre usuarios.** Permitir que varias personas (por ejemplo, los dos miembros de la pareja) gestionen una misma boda de forma compartida.

Estas líneas confirman que la aplicación, además de cumplir los objetivos del trabajo, constituye una base sólida sobre la que seguir evolucionando el producto.

BIBLIOGRAFÍA

Referencias en formato APA (6.^a edición), ordenadas alfabéticamente.

Bodas.net. (s.f.). *Organizar boda*. Recuperado el 11 de agosto de 2026, de <https://www.bodas.net/organizador-bodas>

Docker Inc. (s.f.). *Docker documentation*. Recuperado el 3 de septiembre de 2026, de <https://docs.docker.com>

Growth Market Reports. (2025). *Wedding planning software market research report 2033*. Recuperado el 30 de agosto de 2026, de <https://growthmarketreports.com/report/wedding-planning-software-market>

Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force (IETF). <https://www.rfc-editor.org/rfc/rfc7519>

Meta Open Source. (s.f.). *React*. Recuperado el 11 de agosto de 2026, de <https://react.dev>

Microsoft. (s.f.). *TypeScript*. Recuperado el 11 de agosto de 2026, de <https://www.typescriptlang.org>

OpenAI. (s.f.). *OpenAI API documentation*. Recuperado el 11 de agosto de 2026, de <https://platform.openai.com/docs>

OpenJS Foundation. (s.f.-a). *Express*. Recuperado el 3 de septiembre de 2026, de <https://expressjs.com>

OpenJS Foundation. (s.f.-b). *Node.js*. Recuperado el 3 de septiembre de 2026, de <https://nodejs.org>

PostgreSQL Global Development Group. (s.f.). *PostgreSQL documentation*. Recuperado el 11 de agosto de 2026, de <https://www.postgresql.org/docs/>

Prisma. (s.f.). *Prisma documentation*. Recuperado el 11 de agosto de 2026, de <https://www.prisma.io/docs>

shadcn. (s.f.). *shadcn/ui*. Recuperado el 3 de septiembre de 2026, de <https://ui.shadcn.com>

Tailwind Labs. (s.f.). *Tailwind CSS*. Recuperado el 3 de septiembre de 2026, de <https://tailwindcss.com>

The Knot. (s.f.). *Wedding planner app*. Recuperado el 11 de agosto de 2026, de <https://www.theknot.com/wedding-planning-app>

The Knot Worldwide. (2026). *The Knot Worldwide unveils 2026 Real Weddings Study*. Recuperado el 30 de agosto de 2026, de <https://www.theknotww.com/press-releases/the-knot-worldwide-unveils-2026-real-weddings-study>

Vite. (s.f.). *Vite*. Recuperado el 3 de septiembre de 2026, de <https://vite.dev>

Vitest. (s.f.). *Vitest*. Recuperado el 3 de septiembre de 2026, de <https://vitest.dev>

Zod. (s.f.). *Zod: TypeScript-first schema validation*. Recuperado el 3 de septiembre de 2026, de <https://zod.dev>

Zola. (s.f.). *Wedding planning tools*. Recuperado el 11 de agosto de 2026, de <https://www.zola.com/>

ANEXO A. MANUAL DE INSTALACIÓN

Este anexo describe los pasos necesarios para poner en marcha la aplicación en un entorno local, así como las indicaciones para su despliegue en producción. El proyecto se compone de dos partes independientes —`backend` y `frontend`— que se ejecutan por separado, además de una base de datos PostgreSQL.

A.1. Requisitos previos

Antes de comenzar es necesario tener instalado el siguiente software:

- **Node.js 20 LTS** o superior (incluye `npm`).
- **Docker** (con Docker Compose), para ejecutar la base de datos PostgreSQL. De forma alternativa, puede utilizarse una instalación local de PostgreSQL 16.
- **Git**, para obtener el código fuente.

A.2. Obtención del código

Se clona el repositorio y se accede a la carpeta del proyecto:

```
git clone <URL-del-repositorio>
cd wedding-planner-ai
```

El proyecto contiene dos carpetas principales: `backend` y `frontend`.

A.3. Puesta en marcha del backend

Todos los comandos de este apartado se ejecutan desde la carpeta `backend`.

1. Instalar las dependencias:

```
cd backend
npm install
```

2. Levantar la base de datos. El repositorio incluye un fichero `docker-compose.yml` que arranca un contenedor de PostgreSQL 16 (base de datos `tfm_backend`, usuario y contraseña `postgres`, puerto `5432`):

```
docker compose up -d
```

3. Configurar las variables de entorno. Se crea un fichero `.env` en la carpeta `backend` con las siguientes variables. Las obligatorias son:

Variable	Descripción	Valor de ejemplo
DATABASE_URL	Cadena de conexión a PostgreSQL	postgresql://postgres:postgres@localhost:5432/tfm_backend
JWT_ACCESS_SECRET	Secreto de firma del <i>token</i> de acceso	(cadena larga y aleatoria)
JWT_REFRESH_SECRET	Secreto de firma del <i>token</i> de refresco	(cadena larga y aleatoria)
JWT_ACCESS_EXPIRES	Caducidad del <i>token</i> de acceso	15m
JWT_REFRESH_EXPIRES	Caducidad del <i>token</i> de refresco	7d
PORT	Puerto del servidor	4000
NODE_ENV	Entorno de ejecución	development

De forma **opcional** pueden configurarse:

Variable	Descripción
APP_BASE_URL	URL base del frontend, usada para construir los enlaces de las invitaciones y de recuperación de contraseña (por defecto <code>http://localhost:5173</code>)
SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASS, MAIL_FROM	Credenciales del servidor de correo para el envío de invitaciones. Si no se configuran, el sistema utiliza una cuenta de prueba (Ethereal) que no entrega el correo pero proporciona un enlace de previsualización
CORS_ORIGINS	Lista de orígenes permitidos en producción (separados por comas)
AUTH_RATE_LIMIT_MAX	Número máximo de peticiones a las rutas de autenticación por ventana de tiempo (por defecto 50)
OPENAI_API_KEY	Clave de la API de OpenAI para las asistencias con IA. Si no se define, las funciones de IA se ocultan en la interfaz
OPENAI_MODEL, OPENAI_TIMEOUT_MS	Modelo y tiempo máximo de espera para las llamadas a la IA

4. Generar el cliente de acceso a datos a partir del esquema (necesario tras clonar el repositorio o modificar el esquema):

```
npm run prisma:generate
```

5. Aplicar las migraciones para crear las tablas en la base de datos:

```
npm run prisma:migrate
```

6. (Opcional) Cargar datos de demostración. Este paso crea un usuario de prueba con su boda, útil para explorar la aplicación:

```
npx prisma db seed
```

El usuario de demostración es `demo@planifica2.com` con contraseña `123456`.

7. Arrancar el servidor en modo desarrollo (con recarga automática):

```
npm run dev
```

El backend queda disponible en `http://localhost:4000`, con la API bajo el prefijo `/api`.

A.4. Puesta en marcha del frontend

Los comandos de este apartado se ejecutan desde la carpeta `frontend`.

1. Instalar las dependencias:

```
cd frontend
npm install
```

2. Configurar las variables de entorno. Se copia el fichero de ejemplo y se ajusta si es necesario:

```
cp .env.example .env
```

La variable principal es `VITE_API_URL`, que debe apuntar a la API del backend y terminar en `/api` (por defecto `http://localhost:4000/api`).

3. Arrancar la aplicación en modo desarrollo:

```
npm run dev
```

El frontend queda disponible en `http://localhost:5173`.

A.5. Acceso a la aplicación

Con el backend y el frontend en marcha, se accede desde el navegador a `http://localhost:5173`. Puede registrarse un usuario nuevo o, si se cargaron los datos de demostración, iniciar sesión con las credenciales indicadas en el apartado A.3.

A.6. Despliegue en producción

Para un despliegue en producción, además de configurar `NODE_ENV=production` y unos secretos de sesión seguros (el servidor aborta el arranque si detecta secretos por defecto en producción), se siguen estos pasos:

Backend:

```
npm run build          # compila TypeScript a la carpeta dist/  
npx prisma migrate deploy # aplica las migraciones en producción  
npm start              # ejecuta el servidor compilado
```

Frontend:

```
npm run build          # genera los ficheros estáticos en dist/
```

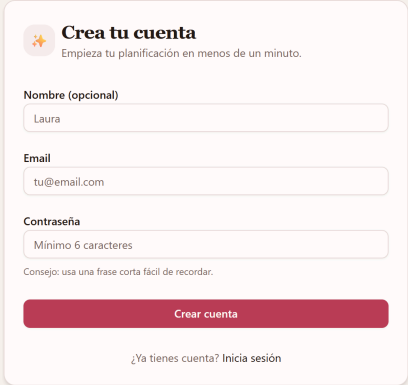
Los ficheros estáticos generados por el frontend se sirven mediante cualquier servidor web estático, apuntando la variable `VITE_API_URL` a la URL pública de la API. En producción conviene restringir los orígenes permitidos mediante `CORS_ORIGINS`.

ANEXO B. MANUAL DE USUARIO

Este anexo describe, apoyándose en capturas de la aplicación, las principales acciones que puede realizar el usuario. Las imágenes corresponden a una boda de demostración con datos representativos.

B.1. Registro e inicio de sesión

Al acceder a la aplicación, el usuario puede crear una cuenta desde la página de registro, indicando su nombre, correo y contraseña. Si ya dispone de cuenta, inicia sesión con su correo y contraseña. Al registrarse se crea automáticamente su primera boda.



Planifica2
TU BODA, BAJO CONTROL

Crea tu cuenta
Empieza tu planificación en menos de un minuto.

Nombre (opcional)
Laura

Email
tu@email.com

Contraseña
Mínimo 6 caracteres

Consejo: usa una frase corta fácil de recordar.

Crear cuenta

¿Ya tienes cuenta? Inicia sesión

Hecho por [Laura](#). El código fuente está disponible en [GitHub](#).

Figura B.1. Página de registro.

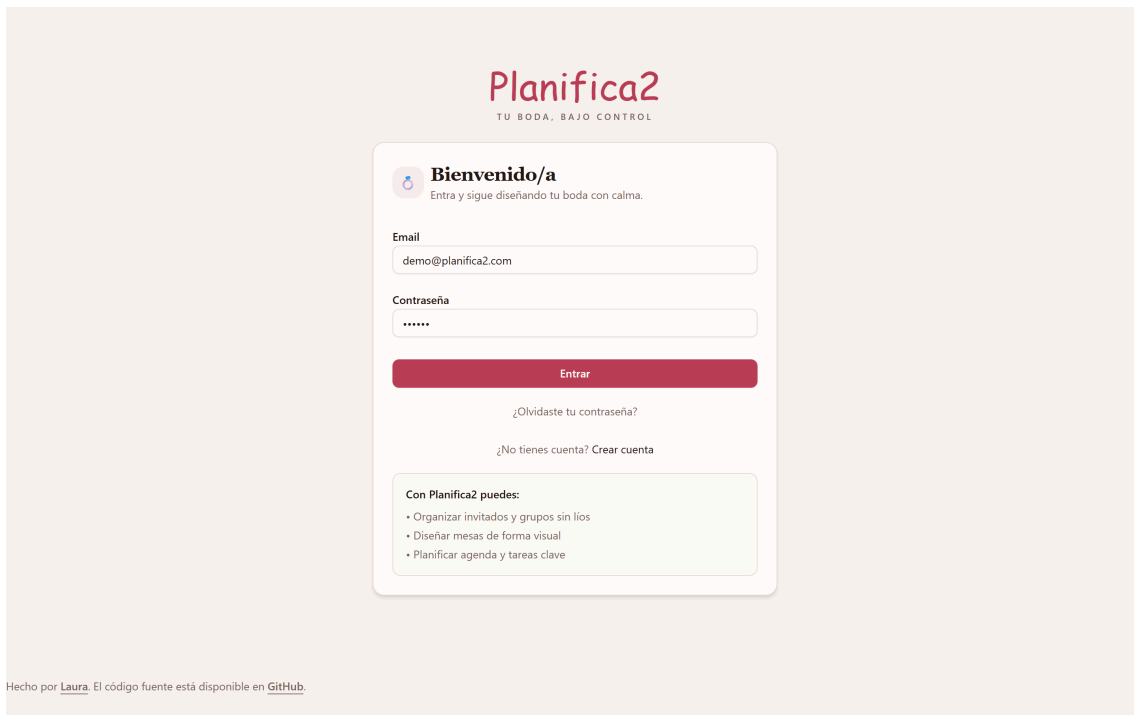


Figura B.2. Página de inicio de sesión.

B.2. Panel de seguimiento

Tras iniciar sesión, el usuario accede al panel de seguimiento, que resume el estado de la boda: invitaciones enviadas, proveedores, progreso de tareas, estado del presupuesto, reparto de invitados por asistencia y próximas tareas. En la cabecera se muestran el selector de boda, la cuenta atrás hasta la fecha del enlace y la campana de avisos.

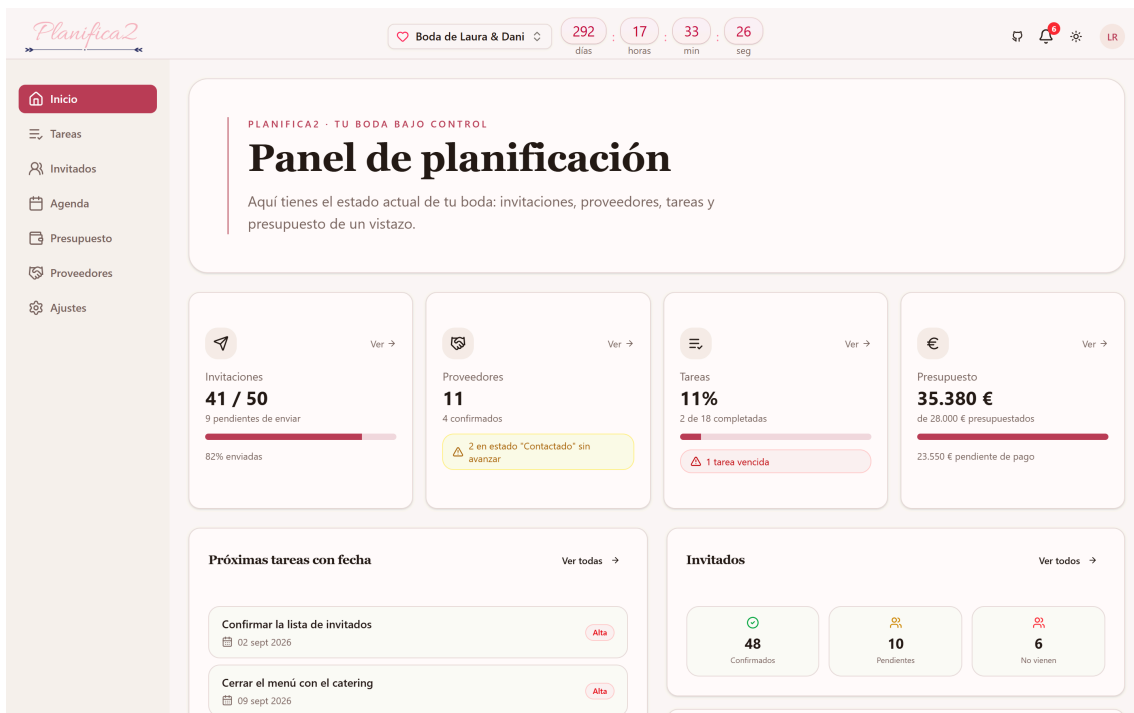


Figura B.3. Panel de seguimiento de la boda.

B.3. Invitados

Desde la pestaña de invitados, el usuario da de alta a los invitados y sus acompañantes, gestiona su estado de asistencia, sus datos y sus alergias, y controla el envío de invitaciones. También puede importar y exportar la lista en formato CSV, y eliminar invitados de forma individual o masiva.

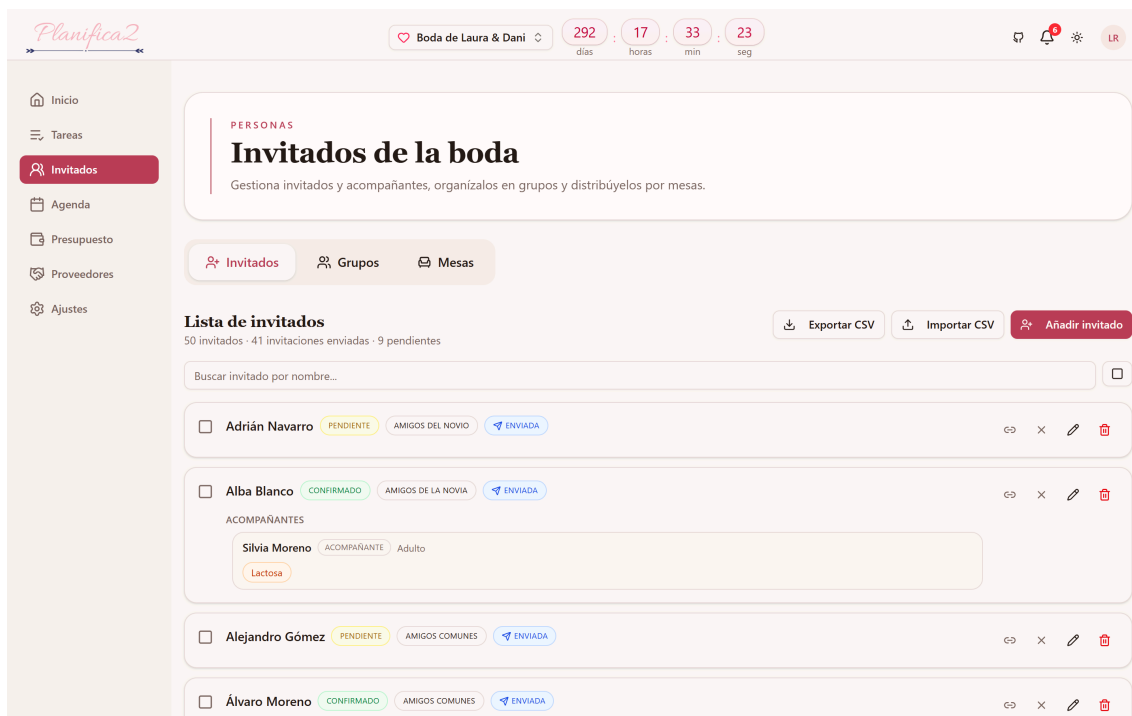


Figura B.4. Lista de invitados con acompañantes.

B.4. Grupos

La pestaña de grupos permite organizar a los invitados en agrupaciones (familia, amigos, compañeros de trabajo, etc.) y asignar invitados a cada grupo.

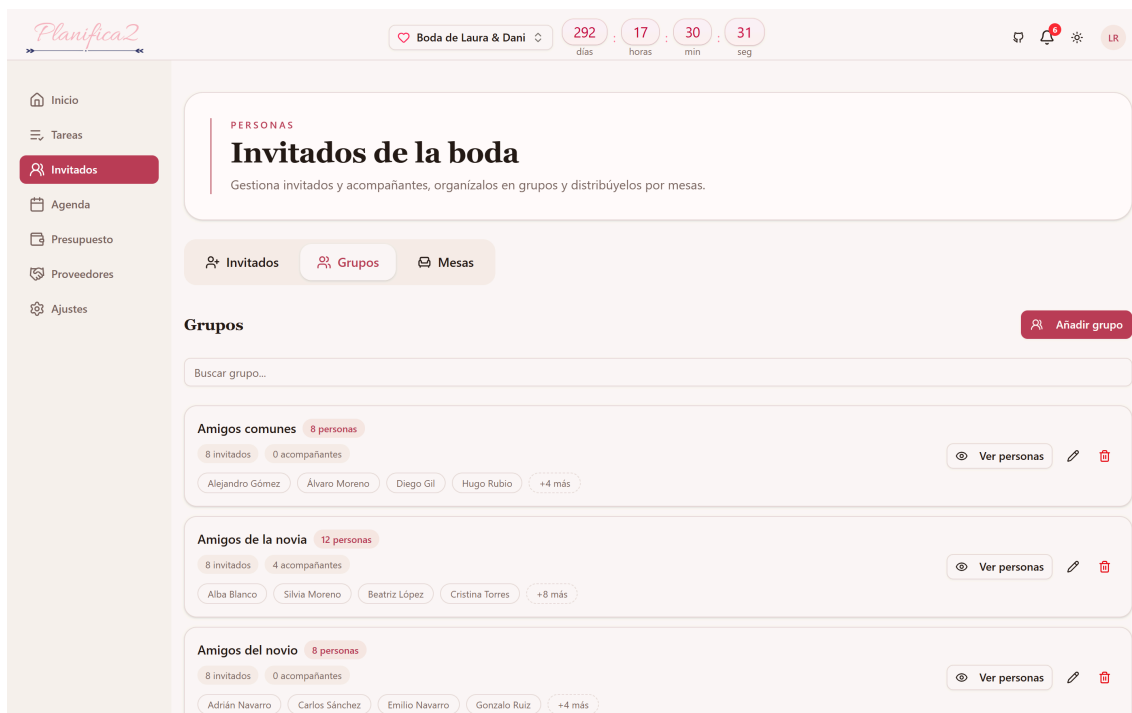


Figura B.5. Gestión de grupos de invitados.

B.5. Mesas

En la pestaña de mesas, el usuario crea las mesas indicando su número de asientos y distribuye a los invitados sobre un mapa visual: selecciona a una persona y la coloca en un asiento libre. Cada mesa muestra su ocupación, y es posible liberar un asiento o vaciar una mesa. Desde aquí puede además solicitarse una distribución automática con inteligencia artificial.

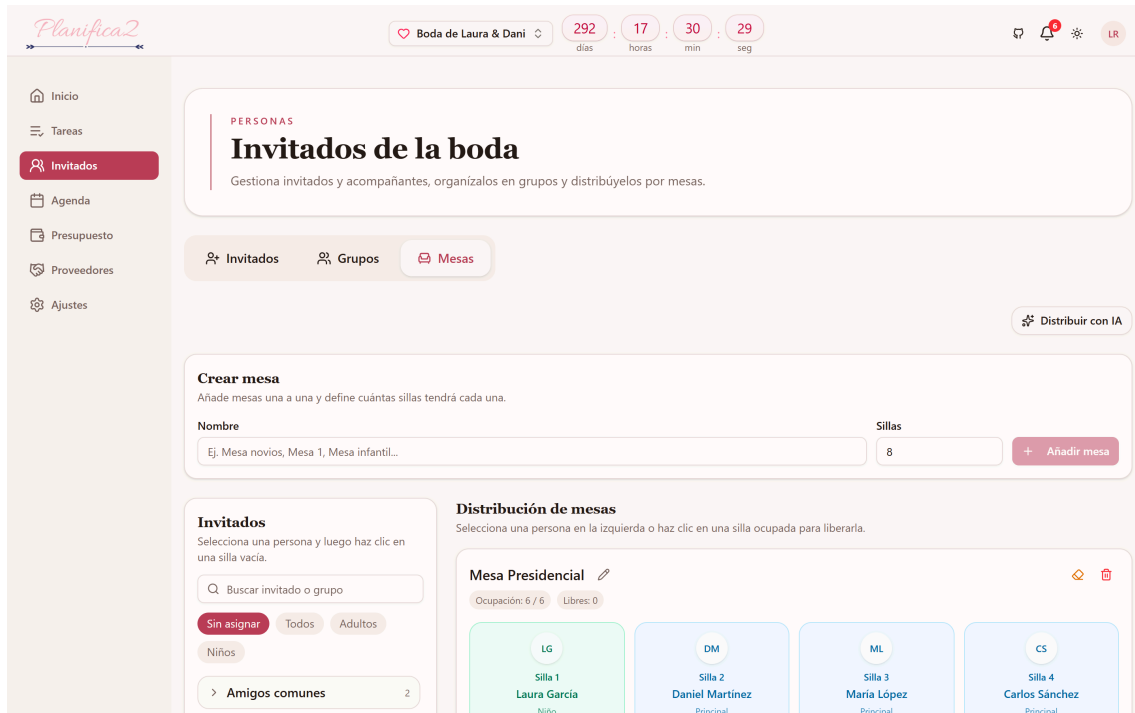


Figura B.6. Distribución de invitados en las mesas.

B.6. Tareas

La sección de tareas permite gestionar el listado de cosas por hacer, cada una con su prioridad, estado, categoría y fecha límite, con filtros para localizarlas rápidamente.

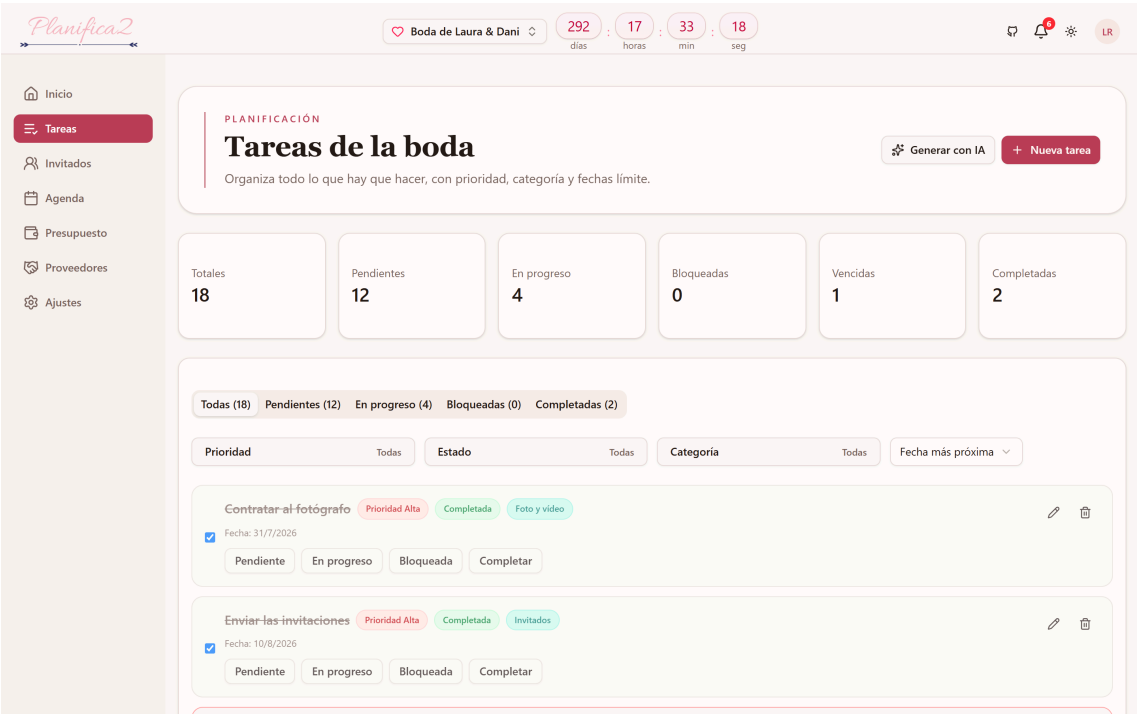


Figura B.7. Gestión de tareas.

B.7. Agenda

La agenda muestra los eventos de la boda (reuniones, pruebas, ceremonia, banquete) en una vista de calendario.

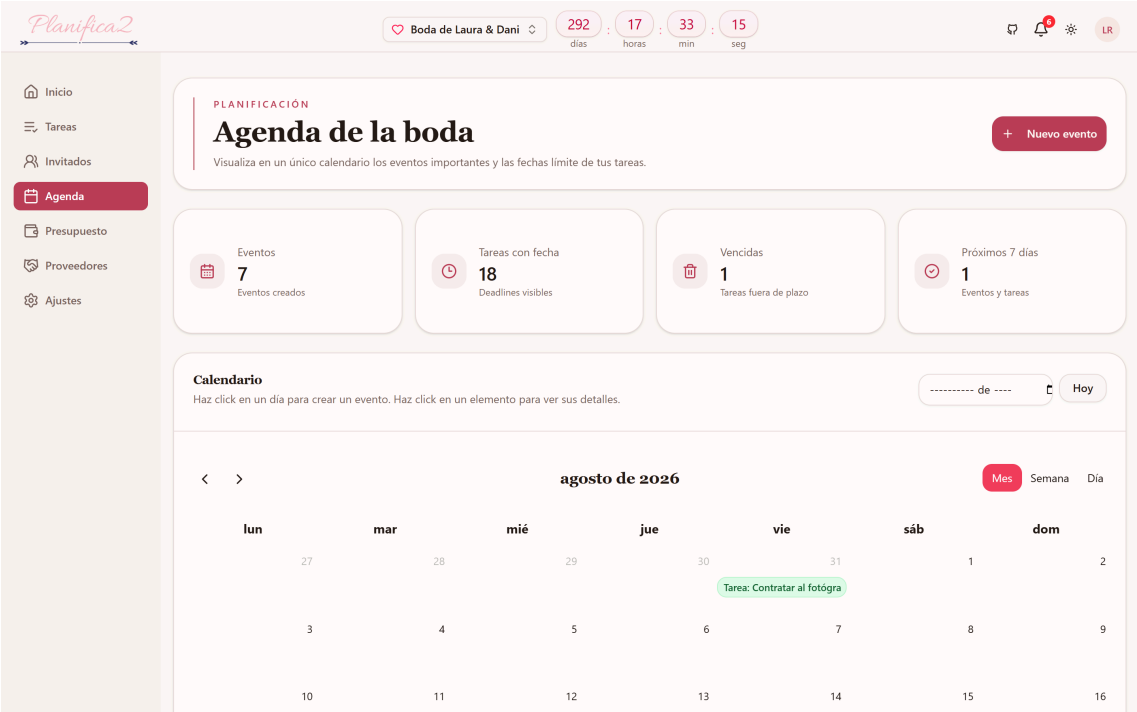


Figura B.8. Agenda de eventos.

B.8. Presupuesto

La sección de presupuesto permite definir el importe global y gestionar las partidas de gasto, con su estado y sus pagos, y visualizar el reparto y el seguimiento mediante

gráficas.

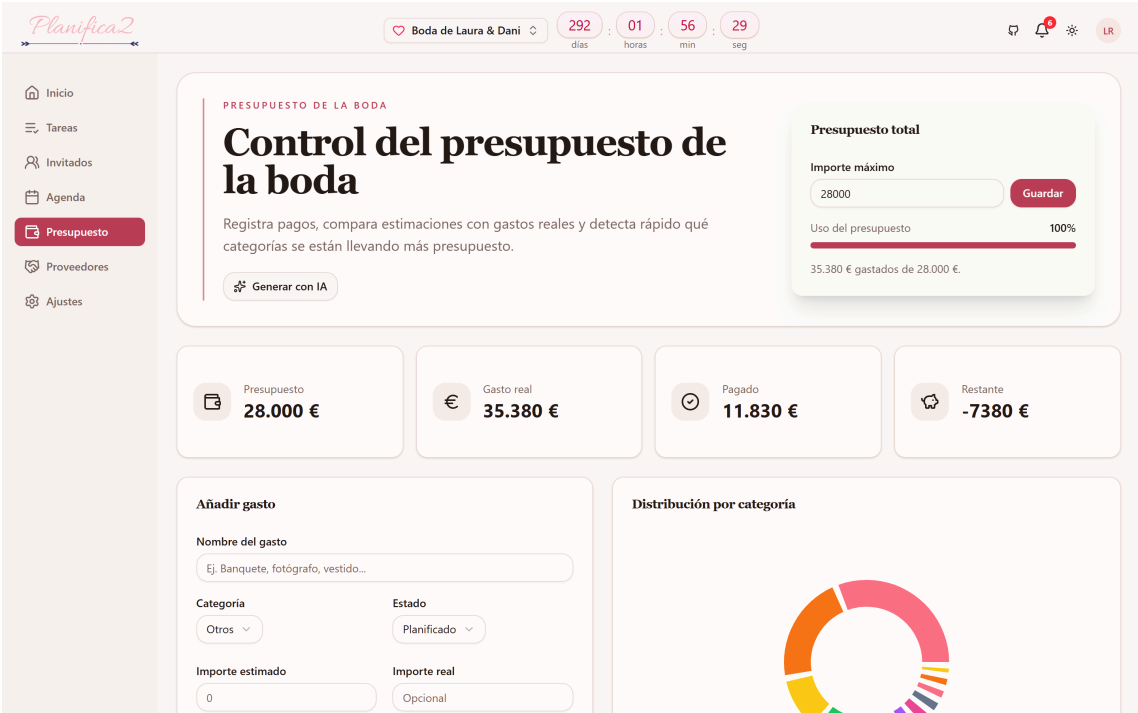


Figura B.9. Gestión del presupuesto.

B.9. Proveedores

Desde la sección de proveedores, el usuario gestiona los proveedores de la boda con su categoría, estado de contratación, datos de contacto y precios, y puede adjuntar documentos a cada uno.

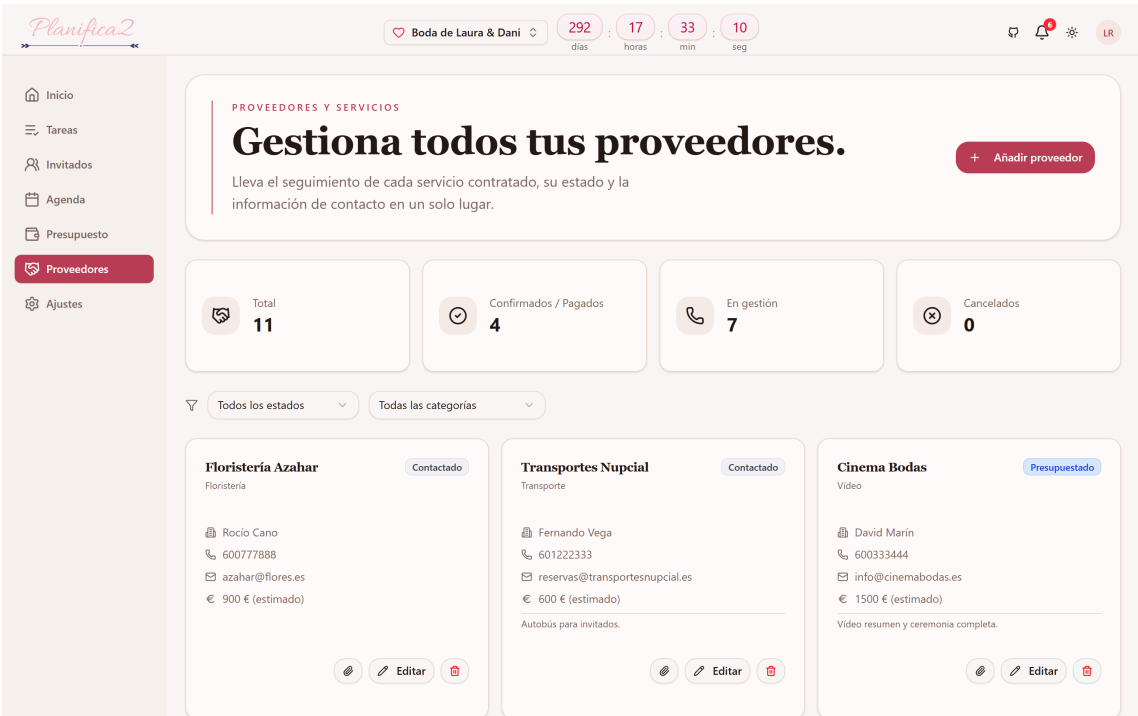


Figura B.10. Gestión de proveedores.

B.10. Ajustes

En la sección de ajustes, el usuario edita los datos de la boda (nombre y fecha) y de su perfil, y puede cambiar su contraseña.

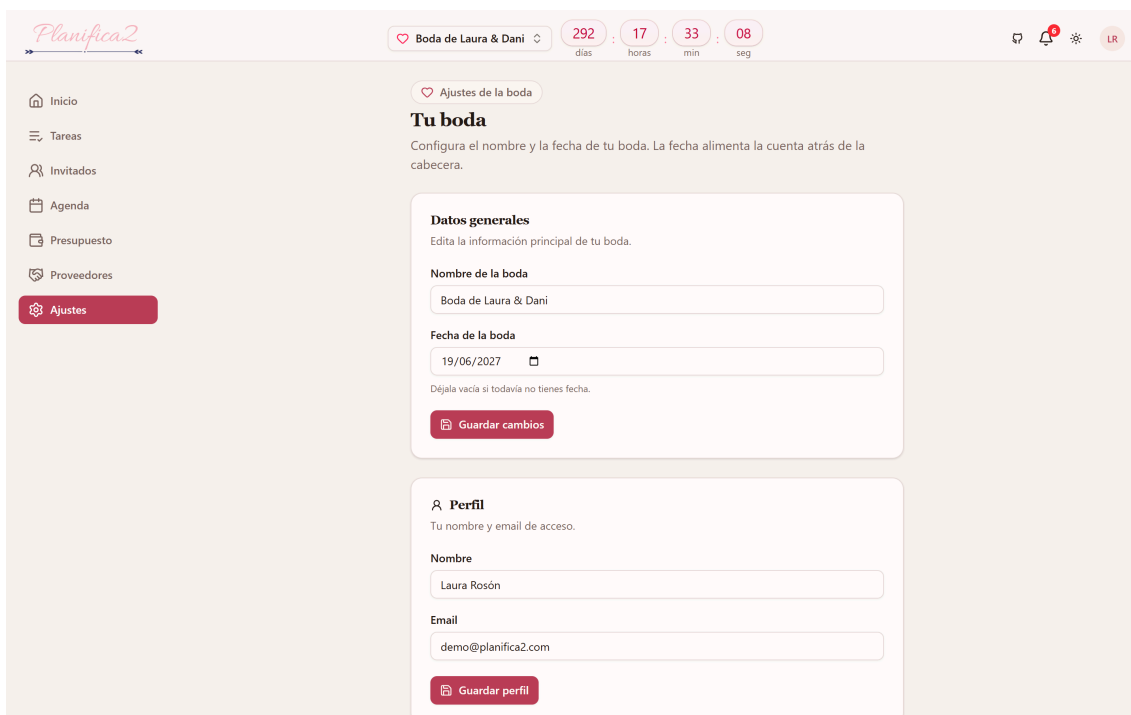


Figura B.11. Ajustes de la boda y del perfil.

B.11. Asistencias con inteligencia artificial

En las secciones de tareas, mesas y presupuesto, el usuario puede solicitar una propuesta generada por inteligencia artificial. El sistema muestra las propuestas en un diálogo donde el usuario revisa, desmarca lo que no desee y confirma; nada se guarda hasta esa confirmación.

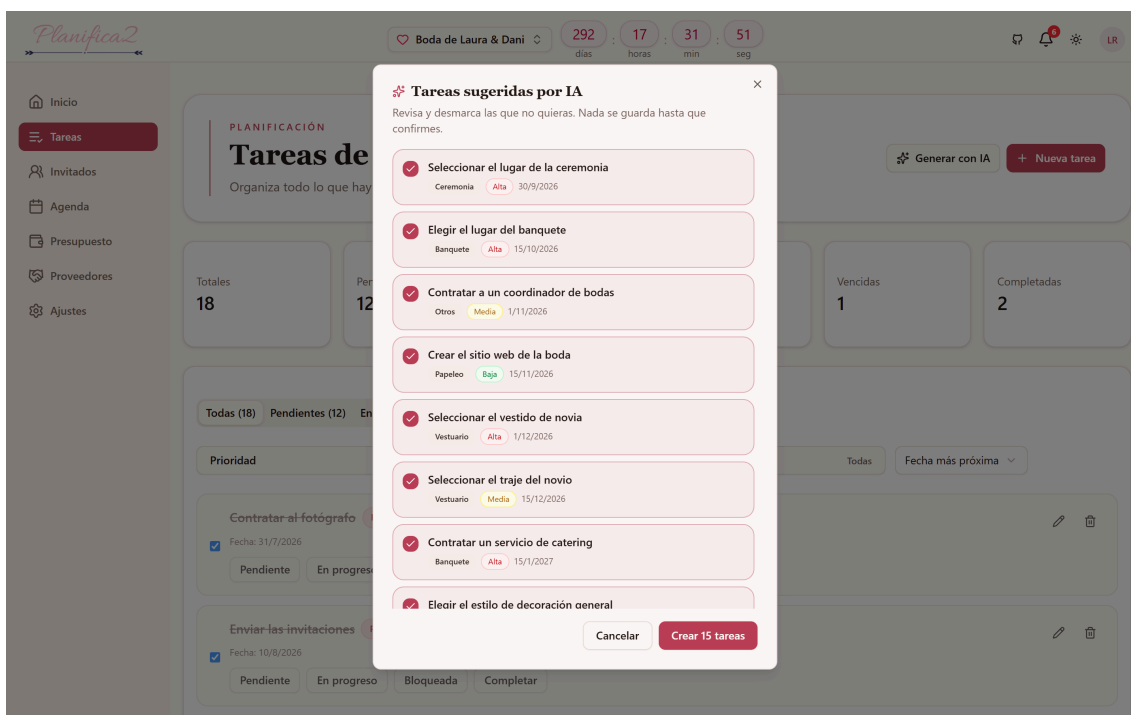


Figura B.12. Propuesta de tareas generada por inteligencia artificial.